

✓ Introduction (Duration: 30 minutes)

Types of Machine Learning

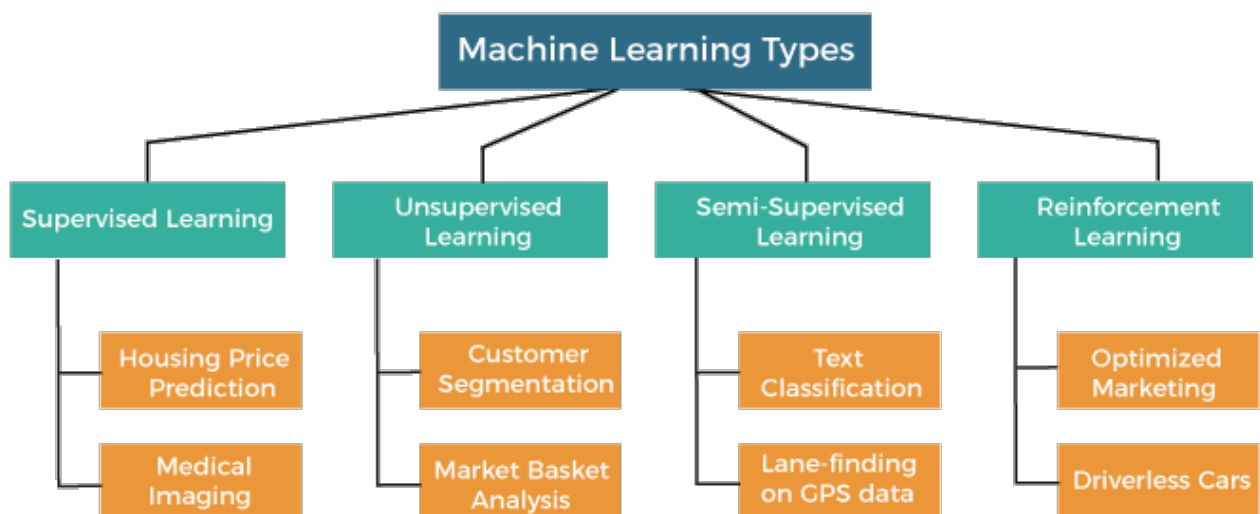
Machine learning is a subset of AI, which enables the machine to automatically learn from data, improve performance from past experiences, and make predictions.

Machine learning contains a set of algorithms that work on a huge amount of data. Data is fed to these algorithms to train them, and on the basis of training, they build the model & perform a specific task.

These ML algorithms help to solve different business problems like Regression, Classification, Forecasting, Clustering, and Associations, etc.

Based on the methods and way of learning, machine learning is divided into mainly four types, which are:

- Supervised Machine Learning
- Unsupervised Machine Learning
- Semi-Supervised Machine Learning
- Reinforcement Learning



<https://www.researchgate.net/profile/Gabriel-Kabanda/publication/371404142/figure/fig1/AS:11431281166457380@1686249390606/Machine-learning-algorithms-and-their-uses.ppm>

Here’s an expanded comparison table including **Supervised ML**, **Unsupervised ML**, **Semi-Supervised ML**, and **Reinforcement ML**:

Criteria	Supervised ML	Unsupervised ML
Definition	Learns using labeled data	Trained using unlabeled data without
Type of Data	Labeled data	Unlabeled data
Type of Problems	Regression and Classification	Association and Clustering
Supervision	Requires extra supervision	No supervision
Algorithms	Linear Regression, Logistic Regression, SVM, KNN, etc.	K-Means, C-Means, Apriori
Aim	Predict outcomes	Discover underlying patterns
Applications	Risk evaluation, Forecasting sales	Recommendation systems, Anomaly

This comprehensive table illustrates how each type of machine learning method differs in definition, data type, problem-solving approach, and real-world applications.

1. Supervised Machine Learning

As its name suggests, Supervised machine learning is based on supervision. It means in the supervised learning technique, we train the machines using the "labelled" dataset, and based on the training, the machine predicts the output. Here, the labelled data specifies that some of the inputs are already mapped to the output. More precisely, we can say; first, we train the machine with the input and corresponding output, and then we ask the machine to predict the output using the test dataset.

Let's understand supervised learning with an example. Suppose we have an input dataset of cats and dog images. So, first, we will provide the training to the machine to understand the images, such as the **shape & size of the tail of cat and dog, Shape of eyes, colour, height (dogs are taller, cats are smaller), etc.** After completion of training, we input the picture of a cat and ask the machine to identify the object and predict the output. Now, the machine is well trained, so it will check all the features of the object, such as height, shape, colour, eyes, ears, tail, etc., and find that it's a cat. So, it will put it in the Cat category. This is the process of how the machine identifies the objects in Supervised Learning.

The main goal of the supervised learning technique is to map the input variable(x) with the output variable(y). Some real-world applications of supervised learning are Risk Assessment, Fraud Detection, Spam filtering, etc.

Categories of Supervised Machine Learning

Supervised machine learning can be classified into two types of problems, which are given below:

- Classification
- Regression

✓ 2. Unsupervised Machine Learning

Unsupervised learning is different from the Supervised learning technique; as its name suggests, there is no need for supervision. It means, in unsupervised machine learning, the machine is trained using the unlabeled dataset, and the machine predicts the output without any supervision.

In unsupervised learning, the models are trained with the data that is neither classified nor labelled, and the model acts on that data without any supervision.

The main aim of the unsupervised learning algorithm is to group or categories the unsorted dataset according to the similarities, patterns, and differences.

Machines are instructed to find the hidden patterns from the input dataset.

Let's take an example to understand it more preciously; suppose there is a basket of fruit images, and we input it into the machine learning model. The images are totally unknown to the model, and the task of the machine is to find the patterns and categories of the objects.

So, now the machine will discover its patterns and differences, such as colour difference, shape difference, and predict the output when it is tested with the test dataset.

Categories of Unsupervised Machine Learning

Unsupervised Learning can be further classified into two types, which are given below:

- Clustering
- Association

1) Clustering

The clustering technique is used when we want to find the inherent groups from the data. It is a way to group the objects into a cluster such that the objects with the most similarities remain in one group and have fewer or no similarities with the objects of other groups. An example of the clustering algorithm is grouping the customers by their purchasing behaviour.

Some of the popular clustering algorithms are given below:

- K-Means Clustering algorithm
- Mean-shift algorithm
- DBSCAN Algorithm
- Principal Component Analysis
- Independent Component Analysis

2) Association

Association rule learning is an unsupervised learning technique, which finds interesting relations among variables within a large dataset. The main aim of this learning algorithm is to find the dependency of one data item on another data item and map those variables accordingly so that it can generate maximum profit. This algorithm is mainly applied in Market Basket analysis, Web usage mining, continuous production, etc.

Some popular algorithms of Association rule learning are Apriori Algorithm, Eclat, FP-growth algorithm.

Advantages and Disadvantages of Unsupervised Learning Algorithm

Advantages:

- These algorithms can be used for complicated tasks compared to the supervised ones because these algorithms work on the unlabeled dataset.

- Unsupervised algorithms are preferable for various tasks as getting the unlabeled dataset is easier as compared to the labelled dataset.

Disadvantages:

- The output of an unsupervised algorithm can be less accurate as the dataset is not labelled, and algorithms are not trained with the exact output in prior.
- Working with Unsupervised learning is more difficult as it works with the unlabelled dataset that does not map with the output.

Applications of Unsupervised Learning

- **Network Analysis:** Unsupervised learning is used for identifying plagiarism and copyright in document network analysis of text data for scholarly articles.
- **Recommendation Systems:** Recommendation systems widely use unsupervised learning techniques for building recommendation applications for different web applications and e-commerce websites.
- **Anomaly Detection:** Anomaly detection is a popular application of unsupervised learning, which can identify unusual data points within the dataset. It is used to discover fraudulent transactions.
- **Singular Value Decomposition:** Singular Value Decomposition or SVD is used to extract particular information from the database. For example, extracting information of each user located at a particular location.

✓ 3. Semi-Supervised Learning

Semi-Supervised learning is a type of Machine Learning algorithm that lies between Supervised and Unsupervised machine learning. It represents the intermediate ground between Supervised (With Labelled training data) and Unsupervised learning (with no labelled training data) algorithms and uses the combination of labelled and unlabeled datasets during the training period.

Although Semi-supervised learning is the middle ground between supervised and unsupervised learning and operates on the data that consists of a few labels, it mostly consists of unlabeled data. As labels are costly, but for corporate purposes, they may have few labels. It is completely different from supervised and unsupervised learning as they are based on the presence & absence of labels.

To overcome the drawbacks of supervised learning and unsupervised learning algorithms, the concept of Semi-supervised learning is introduced. The main aim of semi-supervised learning is to effectively use all the available data, rather than only labelled data like in supervised learning. Initially, similar data is clustered along with an unsupervised learning algorithm, and further, it helps to label the unlabeled data into labelled data. It is because labelled data is a comparatively more expensive acquisition than unlabeled data.

We can imagine these algorithms with an example. Supervised learning is where a student is under the supervision of an instructor at home and college. Further, if that student is self-analysing the same concept without any help from the instructor, it comes under unsupervised learning. Under semi-supervised learning, the student has to revise himself after analyzing the same concept under the guidance of an instructor at college.

Advantages and disadvantages of Semi-supervised Learning

Advantages:

- It is simple and easy to understand the algorithm.
- It is highly efficient.
- It is used to solve drawbacks of Supervised and Unsupervised Learning algorithms.

Disadvantages:

- Iterations results may not be stable.
- We cannot apply these algorithms to network-level data.
- Accuracy is low.

✓ 4. Reinforcement Learning

Reinforcement learning works on a feedback-based process, in which an AI agent (A software component) automatically explore its surrounding by hitting & trail, taking action, learning from experiences, and improving its performance.

Agent gets rewarded for each good action and get punished for each bad action; hence the goal of reinforcement learning agent is to maximize the rewards.

In reinforcement learning, there is no labelled data like supervised learning, and agents learn from their experiences only.

The reinforcement learning process is similar to a human being; for example, a child learns various things by experiences in his day-to-day life. An example of reinforcement learning is to play a game, where the Game is the environment, moves of an agent at each step define states, and the goal of the agent is to get a high score. Agent receives feedback in terms of punishment and rewards.

Due to its way of working, reinforcement learning is employed in different fields such as **Game theory, Operation Research, Information theory, multi-agent systems**.

A reinforcement learning problem can be formalized using **Markov Decision Process(MDP)**. In MDP, the agent constantly interacts with the environment and performs actions; at each action, the environment responds and generates a new state.

Categories of Reinforcement Learning

Reinforcement learning is categorized mainly into two types of methods/algorithms:

- **Positive Reinforcement Learning:** Positive reinforcement learning specifies increasing the tendency that the required behaviour would occur again by adding something. It enhances the strength of the behaviour of the agent and positively impacts it.
- **Negative Reinforcement Learning:** Negative reinforcement learning works exactly opposite to the positive RL. It increases the tendency that the specific behaviour would occur again by avoiding the negative condition.

Real-world Use cases of Reinforcement Learning

- **Video Games:** RL algorithms are much popular in gaming applications. It is used to gain super-human performance. Some popular games that use RL algorithms are **AlphaGO and AlphaGO Zero**.
- **Resource Management:** The "Resource Management with Deep Reinforcement Learning" paper showed that how to use RL in computer to automatically learn and schedule resources to wait for different jobs in order to minimize average job slowdown.
- **Robotics:** RL is widely being used in Robotics applications. Robots are used in the industrial and manufacturing area, and these robots are made more powerful with reinforcement learning. There are different industries that have their vision of building intelligent robots using AI and Machine learning technology.
- **Text Mining** Text-mining, one of the great applications of NLP, is now being implemented with the help of Reinforcement Learning by Salesforce company.

Advantages and Disadvantages of Reinforcement Learning

Advantages

- It helps in solving complex real-world problems which are difficult to be solved by general techniques.
- The learning model of RL is similar to the learning of human beings; hence most accurate results can be found.
- Helps in achieving long term results.

Disadvantage

- RL algorithms are not preferred for simple problems.
- RL algorithms require huge data and computations.
- Too much reinforcement learning can lead to an overload of states which can weaken the results.

The curse of dimensionality limits reinforcement learning for real physical systems.

✓ Unsupervised Machine Learning

In the previous topic, we learned supervised machine learning in which models are trained using labeled data under the supervision of training data. But there may be many cases in which we do not have labeled data and need to find the hidden patterns from the given dataset. So, to solve such types of cases in machine learning, we need unsupervised learning techniques.

What is Unsupervised Learning?

As the name suggests, unsupervised learning is a machine learning technique in which models are not supervised using training dataset. Instead, models itself find the hidden patterns and insights from the given data. It can be compared to learning which takes place in the human brain while learning new things. It can be defined as:

Unsupervised learning is a type of machine learning in which models are trained using unlabeled dataset and are allowed to act on that data without any supervision.

Unsupervised learning cannot be directly applied to a regression or classification problem because unlike supervised learning, we have the input data but no corresponding output data. The goal of unsupervised learning is to **find the underlying structure of dataset, group that data according to similarities, and represent that dataset in a compressed format.**

Example: Suppose the unsupervised learning algorithm is given an input dataset containing images of different types of cats and dogs. The algorithm is never trained upon the given dataset, which means it does not have any idea about the features of the dataset. The task of the unsupervised learning algorithm is to identify the image features on their own. Unsupervised learning algorithm will perform this task by clustering the image dataset into the groups according to similarities between images.



<https://tutorialforbeginner.com/images/tutorial/unsupervised-machine-learning.png>

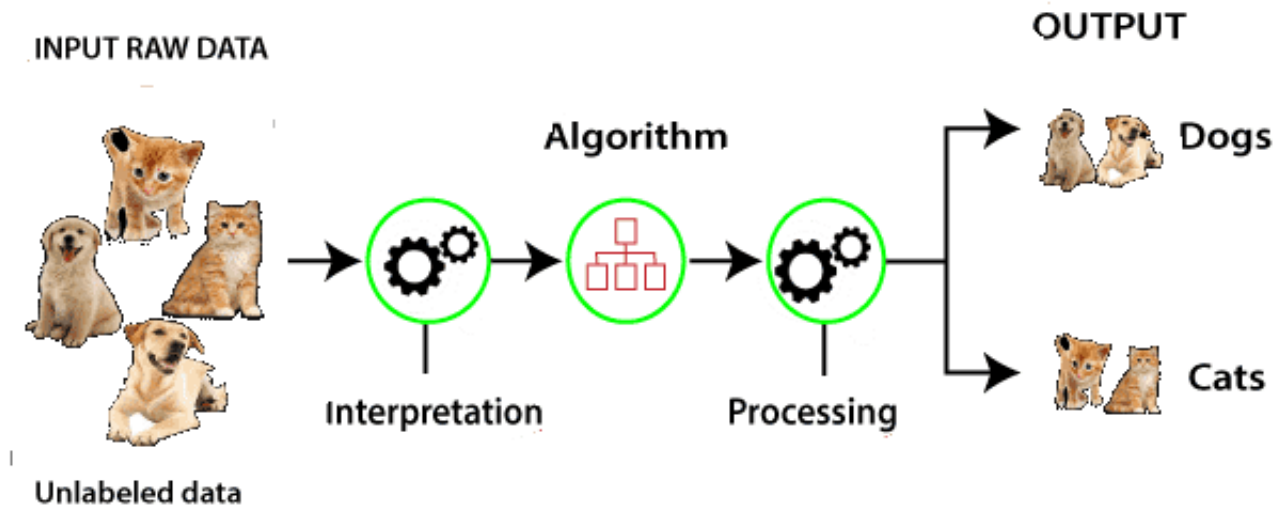
✓ Why use Unsupervised Learning?

Below are some main reasons which describe the importance of Unsupervised Learning:

- Unsupervised learning is helpful for finding useful insights from the data.
- Unsupervised learning is much similar as a human learns to think by their own experiences, which makes it closer to the real AI.
- Unsupervised learning works on unlabeled and uncategorized data which make unsupervised learning more important.
- In real-world, we do not always have input data with the corresponding output so to solve such cases, we need unsupervised learning.

Working of Unsupervised Learning

Working of unsupervised learning can be understood by the below diagram:



<https://tutorialforbeginner.com/images/tutorial/unsupervised-machine-learning-1.png>

Here, we have taken an unlabeled input data, which means it is not categorized and corresponding outputs are also not given. Now, this unlabeled input data is fed to the machine learning model in order to train it. Firstly, it will interpret the raw data to find the hidden patterns from the data and then will apply suitable algorithms such as k-means clustering, Decision tree, etc.

Once it applies the suitable algorithm, the algorithm divides the data objects into groups according to the similarities and difference between the objects.

✓ Types of Unsupervised Learning Algorithm:

The unsupervised learning algorithm can be further categorized into two types of problems:

- **Clustering:** Clustering is a method of grouping the objects into clusters such that objects with most similarities remains into a group and has less or no similarities with the objects of another group. Cluster analysis finds the commonalities between the data objects and categorizes them as per the presence and absence of those commonalities.
- **Association:** An association rule is an unsupervised learning method which is used for finding the relationships between variables in the large database. It determines the set of items that occurs together in the dataset. Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item. A typical example of Association rule is Market Basket Analysis.

Unsupervised Learning algorithms:

Below is the list of some popular unsupervised learning algorithms:

- K-means clustering
- KNN (k-nearest neighbors)
- Hierarchical clustering
- Anomaly detection
- Neural Networks
- Principle Component Analysis
- Independent Component Analysis
- Apriori algorithm
- Singular value decomposition

Advantages of Unsupervised Learning

- Unsupervised learning is used for more complex tasks as compared to supervised learning because, in unsupervised learning, we don't have labeled input data.
- Unsupervised learning is preferable as it is easy to get unlabeled data in comparison to labeled data.

Disadvantages of Unsupervised Learning

- Unsupervised learning is intrinsically more difficult than supervised learning as it does not have corresponding output.
- The result of the unsupervised learning algorithm might be less accurate as input data is not labeled, and algorithms do not know the exact output in advance.

Double-click (or enter) to edit

✓ Clustering in Machine Learning

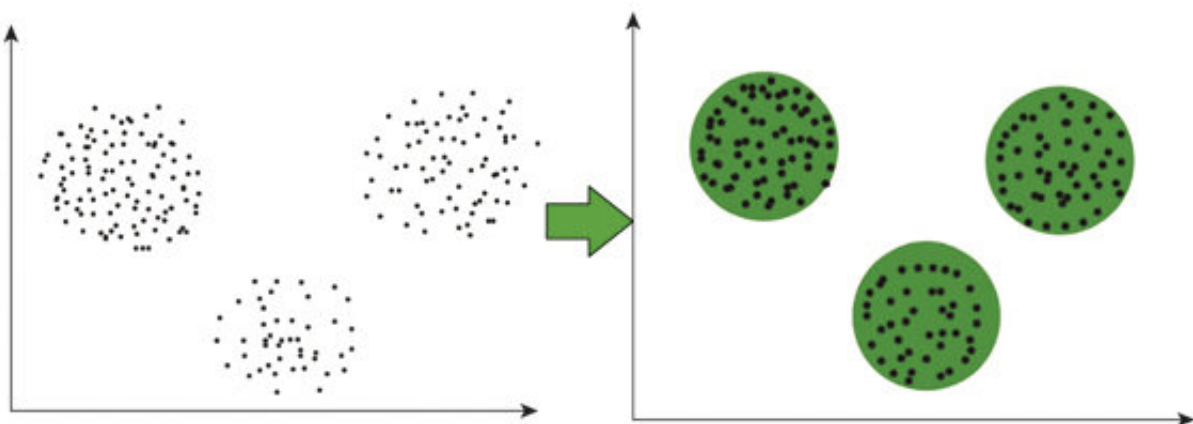
In real world, not every data we work upon has a target variable. This kind of data cannot be analyzed using supervised learning algorithms. We need the help of unsupervised algorithms. One of the most popular type of analysis under unsupervised learning is Cluster analysis. When the goal is to group similar data points in a dataset, then we use cluster analysis. In practical situations, we can use cluster analysis for customer segmentation for targeted advertisements, or in medical imaging to find unknown or new infected areas and many more use cases that we will discuss further in this text.

What is Clustering ?

The task of grouping data points based on their similarity with each other is called Clustering or Cluster Analysis. This method is defined under the branch of Unsupervised Learning, which aims at gaining insights from unlabelled data points, that is, unlike supervised learning we don't have a target variable.

Clustering aims at forming groups of homogeneous data points from a heterogeneous dataset. It evaluates the similarity based on a metric like Euclidean distance, Cosine similarity, Manhattan distance, etc. and then group the points with highest similarity score together.

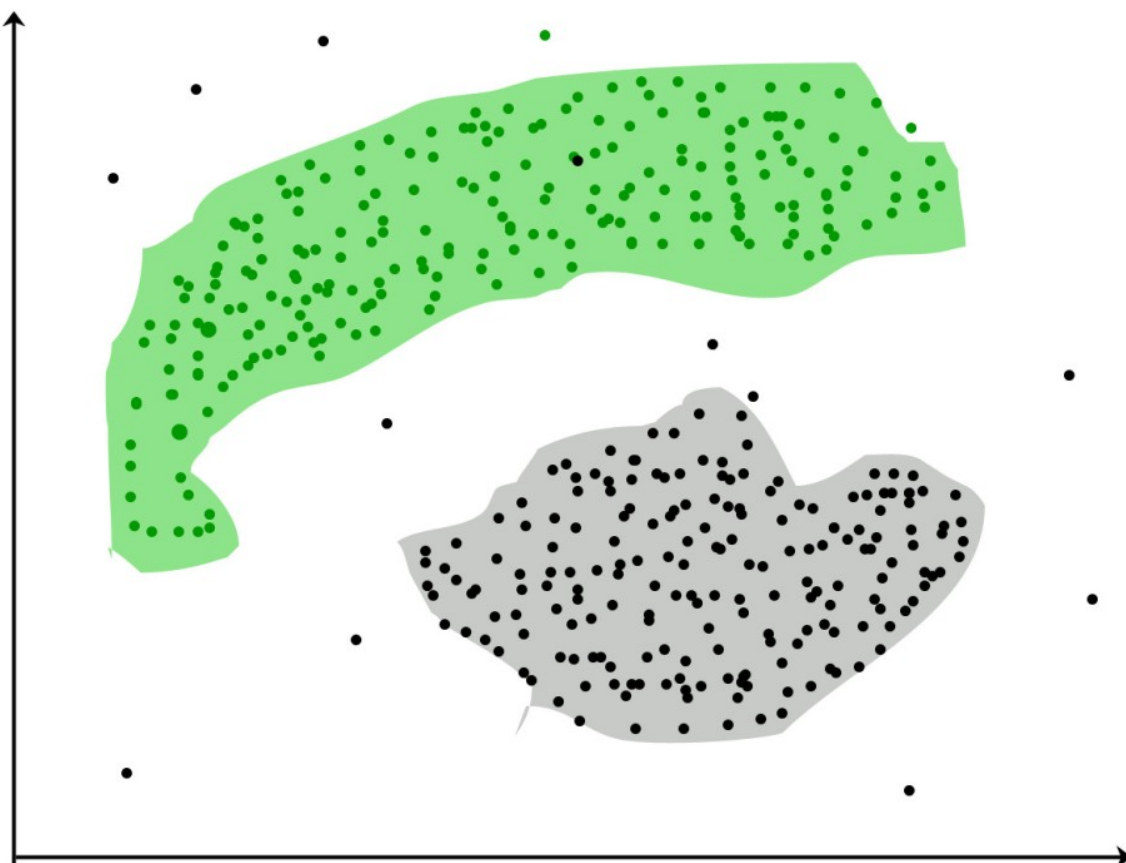
For Example, In the graph given below, we can clearly see that there are 3 circular clusters forming on the basis of distance.



<https://www.researchgate.net/publication/362015383/figure/fig1/AS:11431281170642383@1687843204433/Clustering-algorithms-for-economy-data-based-on-mixture-fuzzy-modelling.jpg>

Now it is not necessary that the clusters formed must be circular in shape. The shape of clusters can be arbitrary. There are many algorithms that work well with detecting arbitrary shaped clusters.

For example, In the below given graph we can see that the clusters formed are not circular in shape.



https://media.licdn.com/dms/image/v2/C4E12AQGfAolchCbqXg/article-cover_image-shrink_720_1280/article-cover_image-shrink_720_1280/0/1605330357602?e=1735776000&v=beta&t=2GxXLNj2ivPx5NPW2cAizAE3Q7OnxKsdt4TYAgZkr8

Types of Clustering Broadly speaking, there are 2 types of clustering that can be

performed to group similar data points:

Hard Clustering: In this type of clustering, each data point belongs to a cluster completely or not. For example, Let's say there are 4 data point and we have to cluster them into 2 clusters. So each data point will either belong to cluster 1 or cluster 2.

Data Points	Clusters
A	C1
B	C2
C	C2
D	C1

Soft Clustering: In this type of clustering, instead of assigning each data point into a separate cluster, a probability or likelihood of that point being that cluster is evaluated. For example, Let's say there are 4 data point and we have to cluster them into 2 clusters. So we will be evaluating a probability of a data point belonging to both clusters. This probability is calculated for all data points.

Data Points	Probability of C1	Probability of C2
A	0.91	0.09
B	0.3	0.7
C	0.17	0.83
D	1	0

Uses of Clustering

Now before we begin with types of clustering algorithms, we will go through the use cases of Clustering algorithms. Clustering algorithms are majorly used for:

- **Market Segmentation** – Businesses use clustering to group their customers and use targeted advertisements to attract more audience.
- **Market Basket Analysis** – Shop owners analyze their sales and figure out which items are majorly bought together by the customers. For example, In USA, according to a study diapers and beers were usually bought together by fathers.
- **Social Network Analysis** – Social media sites use your data to understand your browsing behaviour and provide you with targeted friend recommendations or content recommendations.

- Medical Imaging – Doctors use Clustering to find out diseased areas in diagnostic images like X-rays.
- Anomaly Detection – To find outliers in a stream of real-time dataset or forecasting fraudulent transactions we can use clustering to identify them.
- Simplify working with large datasets – Each cluster is given a cluster ID after clustering is complete. Now, you may reduce a feature set's whole feature set into its cluster ID. Clustering is effective when it can represent a complicated case with a straightforward cluster ID. Using the same principle, clustering data can make complex datasets simpler.

There are many more use cases for clustering but there are some of the major and common use cases of clustering. Moving forward we will be discussing Clustering Algorithms that will help you perform the above tasks.

Types of Clustering Algorithms

At the surface level, clustering helps in the analysis of unstructured data. Graphing, the shortest distance, and the density of the data points are a few of the elements that influence cluster formation. Clustering is the process of determining how related the objects are based on a metric called the similarity measure. Similarity metrics are easier to locate in smaller sets of features. It gets harder to create similarity measures as the number of features increases. Depending on the type of clustering algorithm being utilized in data mining, several techniques are employed to group the data from the datasets.

Various types of clustering algorithms are:

- Centroid-based Clustering (Partitioning methods)
- Density-based Clustering (Model-based methods)
- Connectivity-based Clustering (Hierarchical clustering)
- Distribution-based Clustering

✓ 1. Centroid-based Clustering (Partitioning methods)

Partitioning methods are the most easiest clustering algorithms. They group data points on the basis of their closeness. Generally, the similarity measure chosen for these algorithms are Euclidian distance, Manhattan Distance or Minkowski Distance. The datasets are separated into a predetermined number of clusters, and each cluster is referenced by a vector of values. When compared to the vector value, the input data variable shows no difference and joins the cluster.

! The primary drawback ! for these algorithms is the requirement that we establish the number of clusters, "k," either intuitively or scientifically (using the Elbow Method) before any clustering machine learning system starts allocating the data points. Despite this, it is still the most popular type of clustering. K-means and K-medoids clustering are some examples of this type clustering.

✓ K-Means Clustering

Description

K-Means is an unsupervised machine learning algorithm used for clustering. It divides a dataset into (k) distinct clusters by minimizing the variance within each cluster. The algorithm assigns data points to clusters based on their proximity to the nearest cluster centroid.

Advantages

- **Simple and fast:** Easy to implement and computationally efficient for moderate-sized datasets.
- **Scalable:** Performs well with large datasets when (k) is appropriately chosen.
- **Intuitive:** Results are easy to interpret and visualize.

Disadvantages

- **Requires (k):** The number of clusters (k) must be specified beforehand, which can be non-trivial.
- **Sensitive to initialization:** Poor centroid initialization can lead to suboptimal results.
- **Assumes spherical clusters:** Struggles with clusters of irregular shapes or varying densities.

```
# Import libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

# Load the Iris dataset
iris = load_iris()
iris_data = pd.DataFrame(iris.data, columns=iris.feature_names)
```

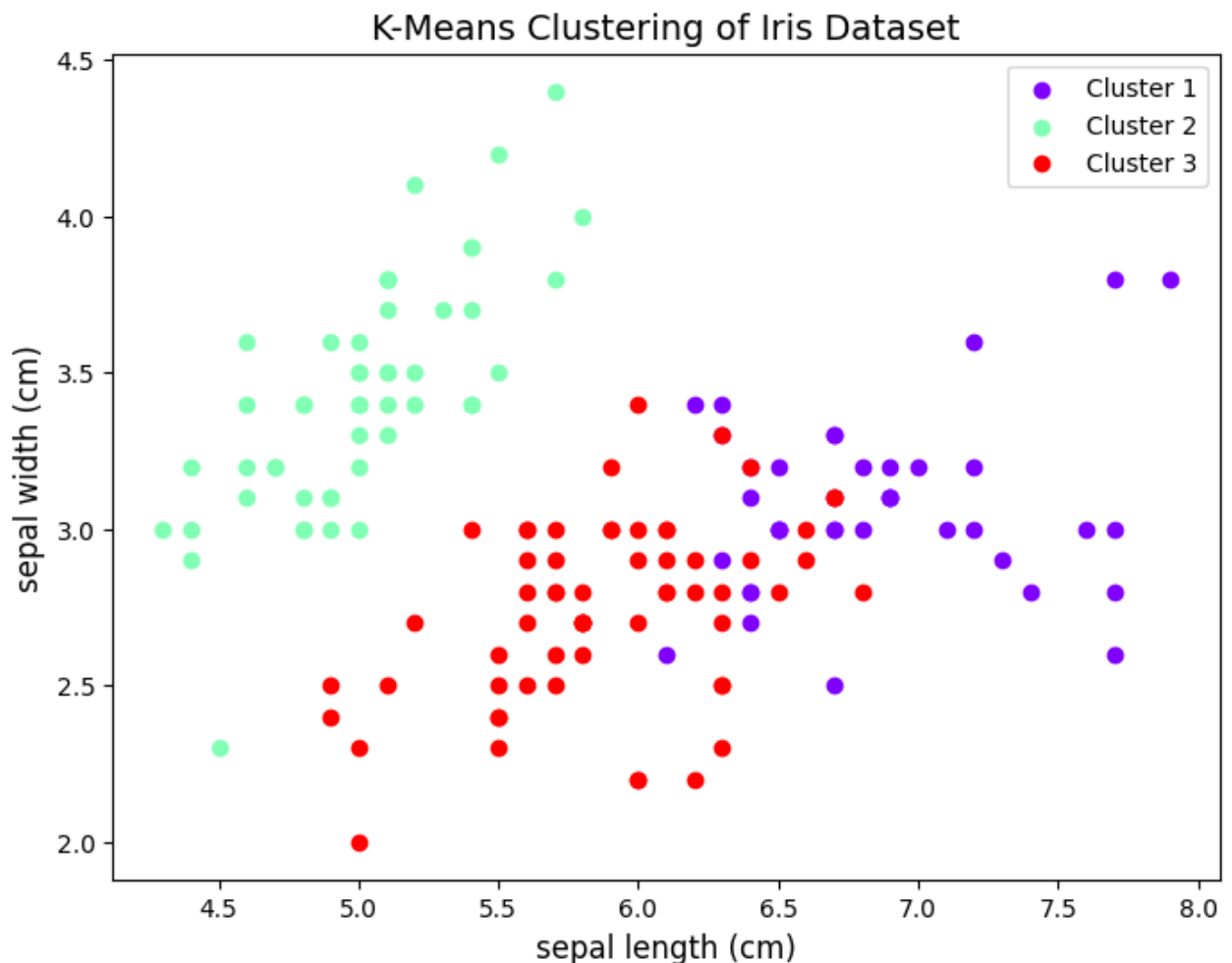
```

# Perform K-Means clustering
kmeans = KMeans(n_clusters=3, random_state=42) # Choosing 3 clusters
iris_data['Cluster'] = kmeans.fit_predict(iris_data)

# Visualize clusters using two features (for simplicity)
plt.figure(figsize=(8, 6))
colors = cm.rainbow(np.linspace(0, 1, 3)) # Define 3 colors for clusters
for i, color in enumerate(colors):
    cluster_data = iris_data[iris_data['Cluster'] == i]
    plt.scatter(cluster_data.iloc[:, 0], cluster_data.iloc[:, 1], label=i)

# Customize the plot
plt.title('K-Means Clustering of Iris Dataset', fontsize=14)
plt.xlabel(iris.feature_names[0], fontsize=12)
plt.ylabel(iris.feature_names[1], fontsize=12)
plt.legend()
plt.show()

```



Explanation of the Example

- **Dataset:** The *Iris dataset* is a classic dataset used for testing clustering and classification methods.
- **Preprocessing:** Directly use numerical features; no scaling is applied for simplicity here.
- **Visualization:** Two dimensions (first two features) are plotted for easy interpretation.

Output

The plot will show three clusters representing groups of data points with similar features, colored differently for clarity.

This exercise introduces students to clustering, helps them visualize results, and understand K-Means' strengths and limitations.

✓ K-Medoids Clustering

Description

K-Medoids is an unsupervised clustering algorithm similar to K-Means but differs in how it defines the center of a cluster. Instead of using the mean as the centroid, K-Medoids selects actual data points (medoids) as cluster centers. It minimizes the sum of dissimilarities (e.g., distance) between data points and their cluster medoid.

Advantages

- **Robust to outliers:** Unlike K-Means, K-Medoids is less sensitive to extreme values because it uses medoids instead of means.
- **Handles non-Euclidean distances:** Works well with various distance metrics (e.g., Manhattan, Cosine).

Disadvantages

- **Computationally intensive:** The pairwise comparison of distances makes it slower than K-Means, especially for large datasets.
- **Choice of (k):** Like K-Means, it requires the number of clusters (k) to be specified.

```
pip install scikit-learn-extra
```

```
Collecting scikit-learn-extra
```

```
  Downloading scikit_learn_extra-0.3.0-cp310-cp310-manylinux_2_17_x86_64.whl (2.0 MB)
Requirement already satisfied: numpy>=1.13.3 in /usr/local/lib/python3.10/site-packages (from scikit-learn-extra)
Requirement already satisfied: scipy>=0.19.1 in /usr/local/lib/python3.10/site-packages (from scikit-learn-extra)
Requirement already satisfied: scikit-learn>=0.23.0 in /usr/local/lib/python3.10/site-packages (from scikit-learn-extra)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/site-packages (from scikit-learn-extra)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/site-packages (from scikit-learn-extra)
  Downloading scikit_learn_extra-0.3.0-cp310-cp310-manylinux_2_17_x86_64.whl (2.0 MB)
2.0/2.0 MB 18.4 MB/s eta 0s
```

```
Installing collected packages: scikit-learn-extra
Successfully installed scikit-learn-extra-0.3.0
```

```
# Import libraries
import pandas as pd
```

```
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from sklearn_extra.cluster import KMedoids
import numpy as np

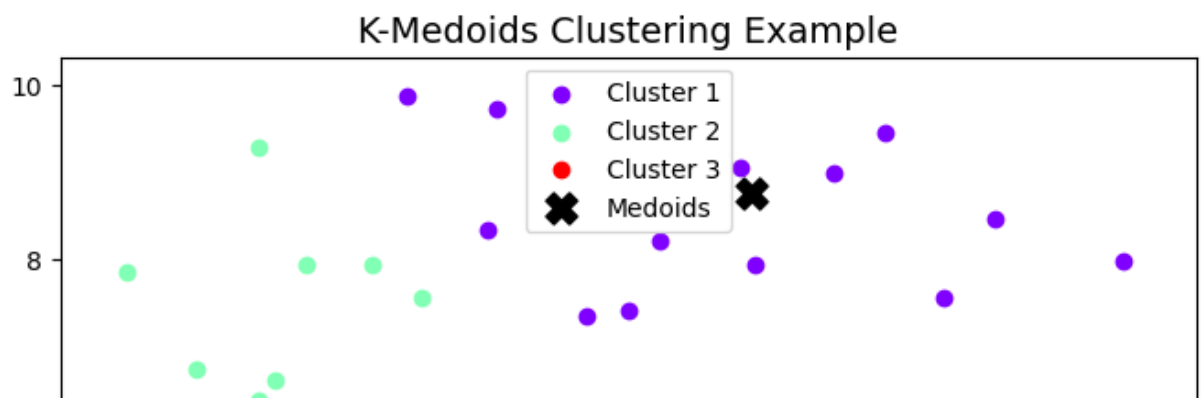
# Create a simple dataset
np.random.seed(42)
data = pd.DataFrame({
    'Feature1': np.random.uniform(1, 10, 50),
    'Feature2': np.random.uniform(1, 10, 50)
})

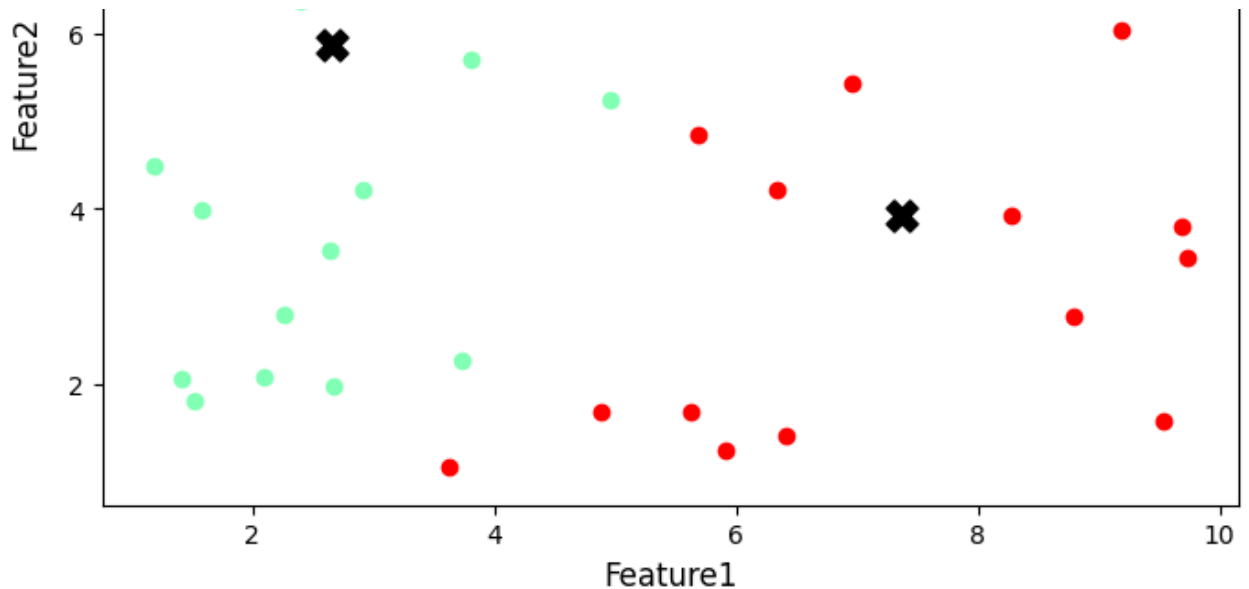
# Apply K-Medoids clustering
kmedoids = KMedoids(n_clusters=3, random_state=42, metric='euclidean')
data['Cluster'] = kmedoids.fit_predict(data)

# Visualize the clusters
plt.figure(figsize=(8, 6))
colors = cm.rainbow(np.linspace(0, 1, 3)) # Define colors for clusters
for i, color in enumerate(colors):
    cluster_data = data[data['Cluster'] == i]
    plt.scatter(cluster_data['Feature1'], cluster_data['Feature2'], c=color)

# Add medoids to the plot
medoid_points = data.iloc[kmedoids.medoid_indices_]
plt.scatter(medoid_points['Feature1'], medoid_points['Feature2'], c='black', s=100)

# Customize the plot
plt.title('K-Medoids Clustering Example', fontsize=14)
plt.xlabel('Feature1', fontsize=12)
plt.ylabel('Feature2', fontsize=12)
plt.legend()
plt.show()
```





✓ Explanation of the Example

- **Dataset:** A small synthetic dataset with two features for simplicity.
- **Clustering Algorithm:** K-Medoids is applied with three clusters.
- **Visualization:** Data points are grouped into clusters, with medoids marked distinctly as black "X" markers.

Output

The plot shows the data divided into three clusters, with medoids clearly identified. The medoids represent the most central data points within each cluster.

This example demonstrates how K-Medoids can create robust clusters and highlights its differences from K-Means.

```
import numpy as np
import matplotlib.pyplot as plt
import time

from sklearn_extra.cluster import KMedoids, CLARA
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
```

```

from sklearn.preprocessing import scale

print(__doc__)

# Authors: Timo Erkkilä <timo.erkkila@gmail.com>
#          Antti Lehmussola <antti.lehmussola@gmail.com>
#          Kornel Kiełczewski <kornel.mail@gmail.com>
# License: BSD 3 clause

np.random.seed(42)

digits = load_digits()
data = scale(digits.data)
n_digits = len(np.unique(digits.target))

reduced_data = PCA(n_components=2).fit_transform(data)

# Step size of the mesh. Decrease to increase the quality of the VQ.
h = 0.02 # point in the mesh [x_min, m_max]x[y_min, y_max].

# Plot the decision boundary. For that, we will assign a color to each
x_min, x_max = reduced_data[:, 0].min() - 1, reduced_data[:, 0].max()
y_min, y_max = reduced_data[:, 1].min() - 1, reduced_data[:, 1].max()
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

plt.figure()
plt.clf()

plt.suptitle(
    "Comparing KMedoids and CLARA",
    fontsize=14,
)

selected_models = [
    (
        KMedoids(metric="cosine", n_clusters=n_digits),
        "KMedoids (cosine)",
    ),
    (
        KMedoids(metric="manhattan", n_clusters=n_digits),
        "KMedoids (manhattan)",
    ),
    (
        CLARA(

```

```

        metric="cosine",
        n_clusters=n_digits,
        init="heuristic",
        n_sampling=50,
    ),
    "CLARA (cosine)",
),
(
    CLARA(
        metric="manhattan",
        n_clusters=n_digits,
        init="heuristic",
        n_sampling=50,
    ),
    "CLARA (manhattan)",
),
]

plot_rows = int(np.ceil(len(selected_models) / 2.0))
plot_cols = 2

for i, (model, description) in enumerate(selected_models):
    # Obtain labels for each point in mesh. Use last trained model.
    init_time = time.time()
    model.fit(reduced_data)
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    computation_time = time.time() - init_time

    # Put the result into a color plot
    Z = Z.reshape(xx.shape)
    plt.subplot(plot_cols, plot_rows, i + 1)
    plt.imshow(
        Z,
        interpolation="nearest",
        extent=(xx.min(), xx.max(), yy.min(), yy.max()),
        cmap=plt.cm.Paired,
        aspect="auto",
        origin="lower",
    )

    plt.plot(
        reduced_data[:, 0], reduced_data[:, 1], "k.", markersize=2, a
    )
    # Plot the centroids as a white X
    centroids = model.cluster_centers_

```

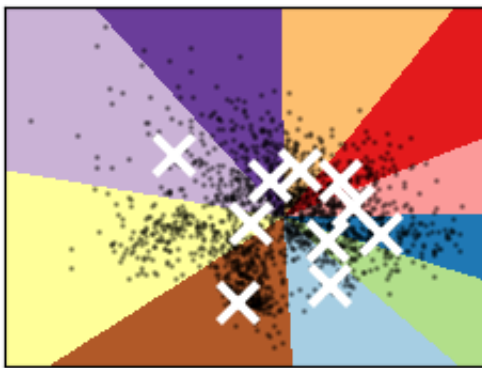
```
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker="x",
    s=169,
    linewidths=3,
    color="w",
    zorder=10,
)
plt.title(description + ": %.2Fs" % (computation_time))
plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)
plt.xticks(())
plt.yticks(())

plt.show()
```

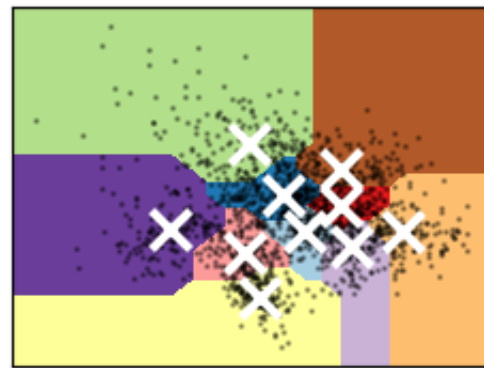
Automatically created module for IPython interactive environment

Comparing KMedoids and CLARA

KMedoids (cosine): 1.35s



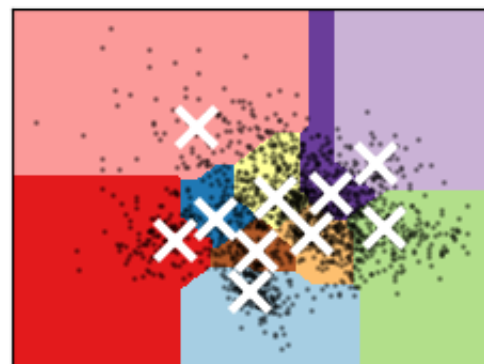
KMedoids (manhattan): 0.73s



CLARA (cosine): 0.32s



CLARA (manhattan): 0.25s



2. Density-based Clustering (Model-based methods)

Density-based clustering, a model-based method, finds groups based on the density of data points. Contrary to centroid-based clustering, which requires that the number of clusters be predefined and is sensitive to initialization, density-based clustering determines the number of clusters automatically and is less susceptible to beginning positions. They are great at handling clusters of different sizes and forms, making them ideally suited for datasets with irregularly shaped or overlapping clusters. These methods manage both dense and sparse data regions by focusing on local density and can distinguish clusters with a variety of morphologies.

In contrast, centroid-based grouping, like k-means, has trouble finding arbitrary shaped clusters. Due to its preset number of cluster requirements and extreme sensitivity to the initial positioning of centroids, the outcomes can vary. Furthermore, the tendency of centroid-based approaches to produce spherical or convex clusters restricts their capacity to handle complicated or irregularly shaped clusters.

! In conclusion, density-based clustering overcomes the drawbacks of centroid-based techniques by autonomously choosing cluster sizes ! , being resilient to initialization, and successfully capturing clusters of various sizes and forms. The most popular density-based clustering algorithm is DBSCAN.

Clustering analysis or simply Clustering is basically an Unsupervised learning method that divides the data points into a number of specific batches or groups, such that the data points in the same groups have similar properties and data points in different groups have different properties in some sense. It comprises many different methods based on differential evolution. E.g. K-Means (distance between points), Affinity propagation (graph distance), Mean-shift (distance between points), DBSCAN (distance between nearest points), Gaussian mixtures (Mahalanobis distance to centers), Spectral clustering (graph distance), etc.

Fundamentally, all clustering methods use the same approach i.e. first we calculate similarities and then we use it to cluster the data points into groups or batches. Here we will focus on the Density-based spatial clustering of applications with noise (DBSCAN) clustering method.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Description

DBSCAN is a density-based clustering algorithm that groups data points that are closely packed together and marks data points in low-density regions as noise or outliers. It uses two parameters:

- **Epsilon (ϵ):** The maximum distance between points to be considered neighbors.
- **MinPts:** The minimum number of points required to form a dense region (a cluster).

Advantages

- **Handles noise:** Effectively identifies and handles outliers.
- **No need to specify clusters (k):** Unlike K-Means, the number of clusters does not need to be predetermined.
- **Detects arbitrary-shaped clusters:** Works well for non-spherical clusters.

Disadvantages

- **Parameter sensitivity:** Performance depends heavily on (ϵ) and MinPts values.
- **Inefficiency with high dimensions:** Can struggle with large, high-dimensional datasets due to computational cost.

```
# Import libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler

# Create synthetic dataset
```



```

np.random.seed(42)
data, labels_true = make_blobs(n_samples=300, centers=4, cluster_std=0.6)

# Scale the data for better performance
data = StandardScaler().fit_transform(data)

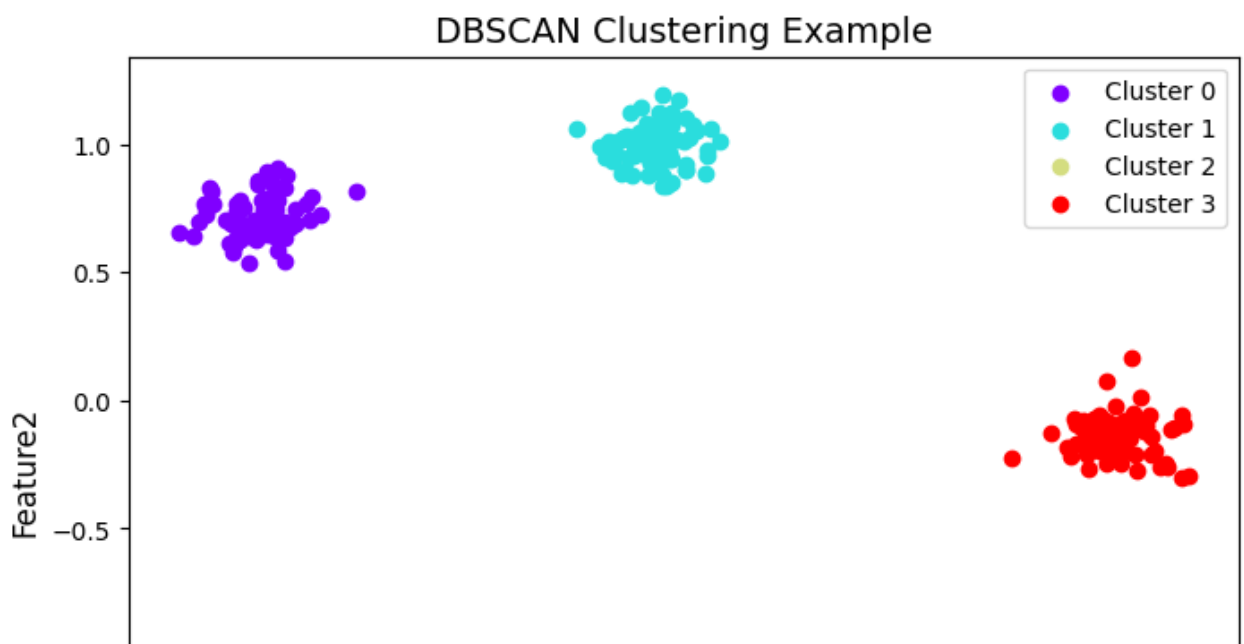
# Apply DBSCAN clustering
dbscan = DBSCAN(eps=0.3, min_samples=5)
clusters = dbscan.fit_predict(data)

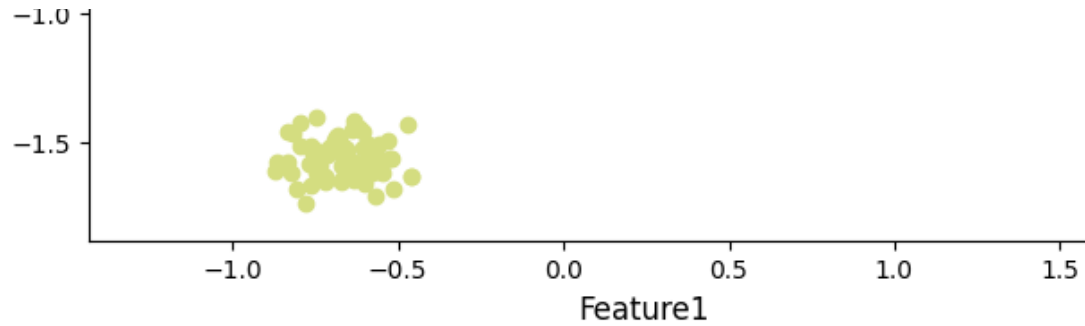
# Convert to DataFrame for visualization
df = pd.DataFrame(data, columns=['Feature1', 'Feature2'])
df['Cluster'] = clusters

# Visualize the clusters
plt.figure(figsize=(8, 6))
unique_clusters = set(clusters)
colors = cm.rainbow(np.linspace(0, 1, len(unique_clusters)))
for cluster, color in zip(unique_clusters, colors):
    cluster_data = df[df['Cluster'] == cluster]
    plt.scatter(cluster_data['Feature1'], cluster_data['Feature2'], label=cluster)

# Customize the plot
plt.title('DBSCAN Clustering Example', fontsize=14)
plt.xlabel('Feature1', fontsize=12)
plt.ylabel('Feature2', fontsize=12)
plt.legend()
plt.show()

```





Explanation of the Example

- **Dataset:** A synthetic dataset with four distinct clusters is created using `make_blobs`.
- **DBSCAN Parameters:**
 - `eps=0.3`: The maximum radius for neighborhood inclusion.
 - `min_samples=5`: The minimum number of points to form a dense region.
- **Visualization:** Points are plotted by cluster, with outliers labeled as "Noise."

Output

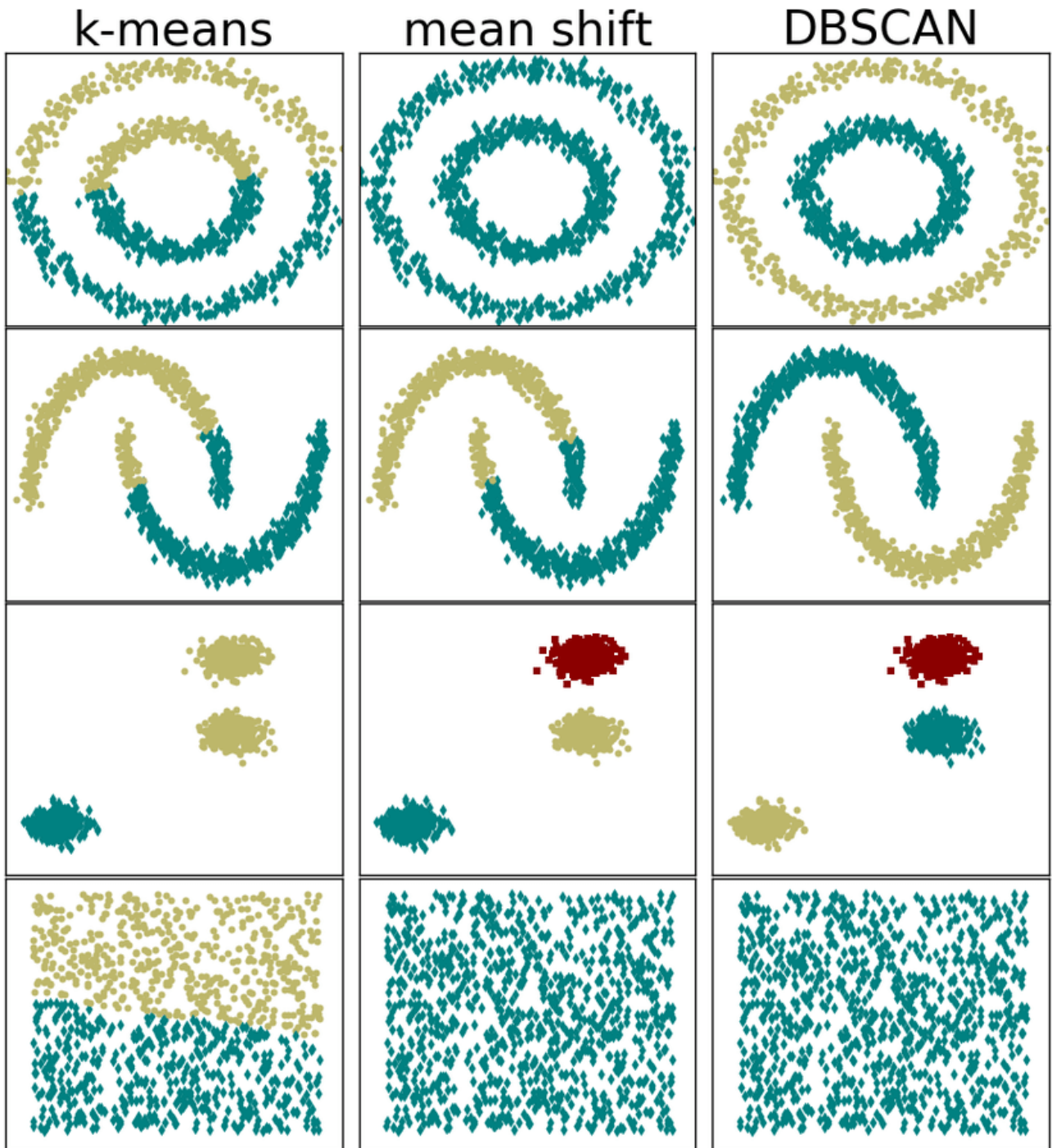
The plot displays clusters detected by DBSCAN, with noise points marked distinctly. This example highlights DBSCAN's ability to handle non-spherical clusters and outliers effectively.

Here is a **comparison table** summarizing the key features of **K-Means**, **K-Medoids**, and **DBSCAN** based on various criteria:

Criteria	K-Means	
Type of Clustering	Centroid-based (mean as center)	Centroid-based (m
Cluster Shape	Assumes spherical clusters	Assumes spherical
Outlier Handling	Sensitive to outliers	Robust to outliers
Scalability	Highly scalable for large datasets	Less scalable due t
Number of Clusters ((k))	Must be specified in advance	Must be specified i
Distance Metric	Typically uses Euclidean distance	Flexible; can use va
Initialization Sensitivity	Sensitive to initial centroids	Less sensitive, as n
Noise and Overlapping Regions	Struggles with noisy data and overlapping clusters	Handles overlappin
Parameter Requirements	Requires (k) (number of clusters)	Requires (k) (numb
Complexity	($O(n \cdot k \cdot i)$), where (i) is the number of iterations	Higher complexity t
Best Use Cases	Large datasets with spherical and balanced clusters	Small to medium d

▼ **Summary**

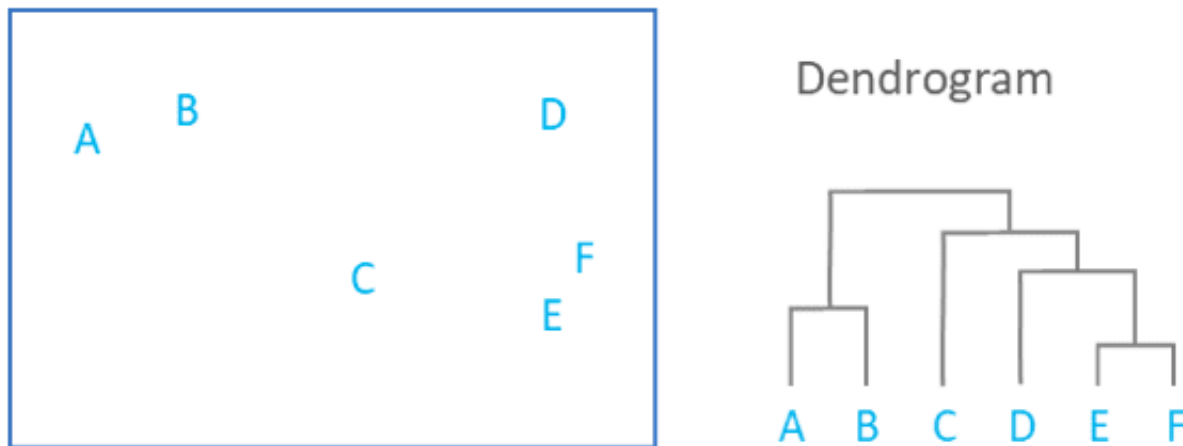
- **K-Means:** Best for large datasets with well-separated, spherical clusters and minimal noise.
- **K-Medoids:** Suitable for small to medium datasets, robust to outliers, and flexible with distance metrics.
- **DBSCAN:** Ideal for datasets with arbitrary-shaped clusters, noise, and varying densities but less efficient for high-dimensional data.



<https://www.researchgate.net/profile/Gordon-Dobie/publication/323502924/figure/fig1/AS:867490678710273@1583837303235/Comparison-of-k-means-Mean-Shift-and-DBSCAN-16.ppm>

✓ 3. Connectivity-based Clustering (Hierarchical clustering)

A dendrogram is a diagram that shows the hierarchical relationship between objects. It is most commonly created as an output from hierarchical clustering. The main use of a dendrogram is to work out the best way to allocate objects to clusters. The dendrogram below shows the hierarchical clustering of six observations shown on the scatterplot to the left. (Dendrogram is often miswritten as dendogram.)



<https://cdn-dfnaj.nitrocdn.com/xxeFXDnBIOflfPsgwjDLywlQwPChAOzV/assets/images/optimized/rev-162f124/www.displayr.com/wp-content/uploads/2018/03/What-is-a-Dendrogram.png>

How to read a dendrogram

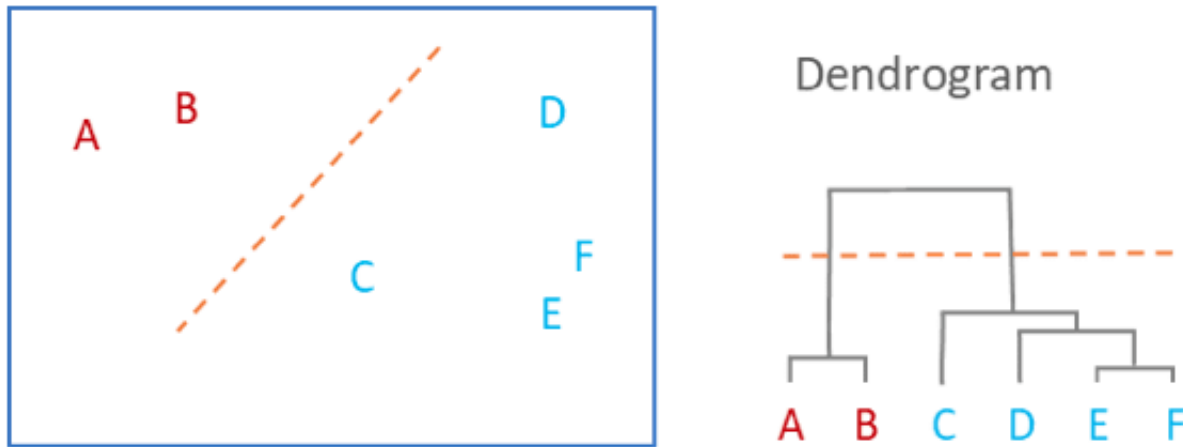
The key to interpreting a dendrogram is to focus on the height at which any two objects are joined together. In the example above, we can see that E and F are most similar, as the height of the link that joins them together is the smallest. The next two most similar objects are A and B.

In the dendrogram above, the height of the dendrogram indicates the order in which the clusters were joined. A more informative dendrogram can be created where the heights reflect the distance between the clusters as is shown below. In this case, the dendrogram shows us that the big difference between clusters is between the cluster of A and B versus that of C, D, E, and F.

It is important to appreciate that the dendrogram is a summary of the distance matrix, and, as occurs with most summaries, information is lost. For example, the dendrogram suggests that C and D are much closer to each other than is C to B, but the original data (shown in the scatterplot), shows us that this is not true. To use some jargon, a dendrogram is only accurate when data satisfies the ultrametric tree inequality, and this is unlikely for any real-world data.

The consequence of the information loss is that the dendrograms are most accurate at the bottom, showing which items are very similar.

Allocating observations to clusters Observations are allocated to clusters by drawing a horizontal line through the dendrogram. Observations that are joined together below the line are in clusters. In the example below, we have two clusters. One cluster combines A and B, and a second cluster combining C, D, E, and F.



<https://cdn-dfnaj.nitrocdn.com/xxeFXDnBIOflfPsgwjDLywlQwPChAOzV/assets/images/optimized/ev-162f124/www.displayr.com/wp-content/uploads/2018/03/Screen-Shot-2018-03-21-at-4.45.46-pm.png>

Dendrograms cannot tell you how many clusters you should have A common mistake people make when reading dendrograms is to assume that the shape of the dendrogram gives a clue as to how many clusters exist. In the example above, the (incorrect) interpretation is that the dendrogram shows that there are two clusters, as the distance between the clusters (the vertical segments of the dendrogram) are highest between two and three clusters.

Interpretation of this kind is justified only when the ultrametric tree inequality holds, which, as mentioned above, is very rare. In general, it is a mistake to use dendrograms as a tool for determining the number of clusters in data. Where there is an obviously “correct” number of clusters this will often be evident in a dendrogram. However, dendrograms often suggest a correct number of clusters when there is no real evidence to support the conclusion.

✓ Description

Hierarchical clustering is an unsupervised learning algorithm that builds a hierarchy of clusters. It can be performed in two ways:

- **Agglomerative (Bottom-Up):** Each data point starts as its own cluster, and clusters are iteratively merged based on similarity.
- **Divisive (Top-Down):** All points start in a single cluster, which is recursively split into smaller clusters.

Clusters are often visualized using a **dendrogram**, which shows the merging or splitting process.

Advantages

- **No need to specify (k):** Unlike K-Means, you don't need to predefine the number of clusters.
- **Versatile:** Can capture clusters of varying shapes and sizes.
- **Dendrogram:** Provides a clear visualization of the cluster hierarchy.

Disadvantages

- **Scalability:** Computationally intensive for large datasets.
- **Sensitive to noise:** Can be affected by outliers and noisy data.
- **Distance metric dependence:** Results depend heavily on the chosen linkage and distance metric (e.g., Euclidean, Manhattan).

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.datasets import make_blobs

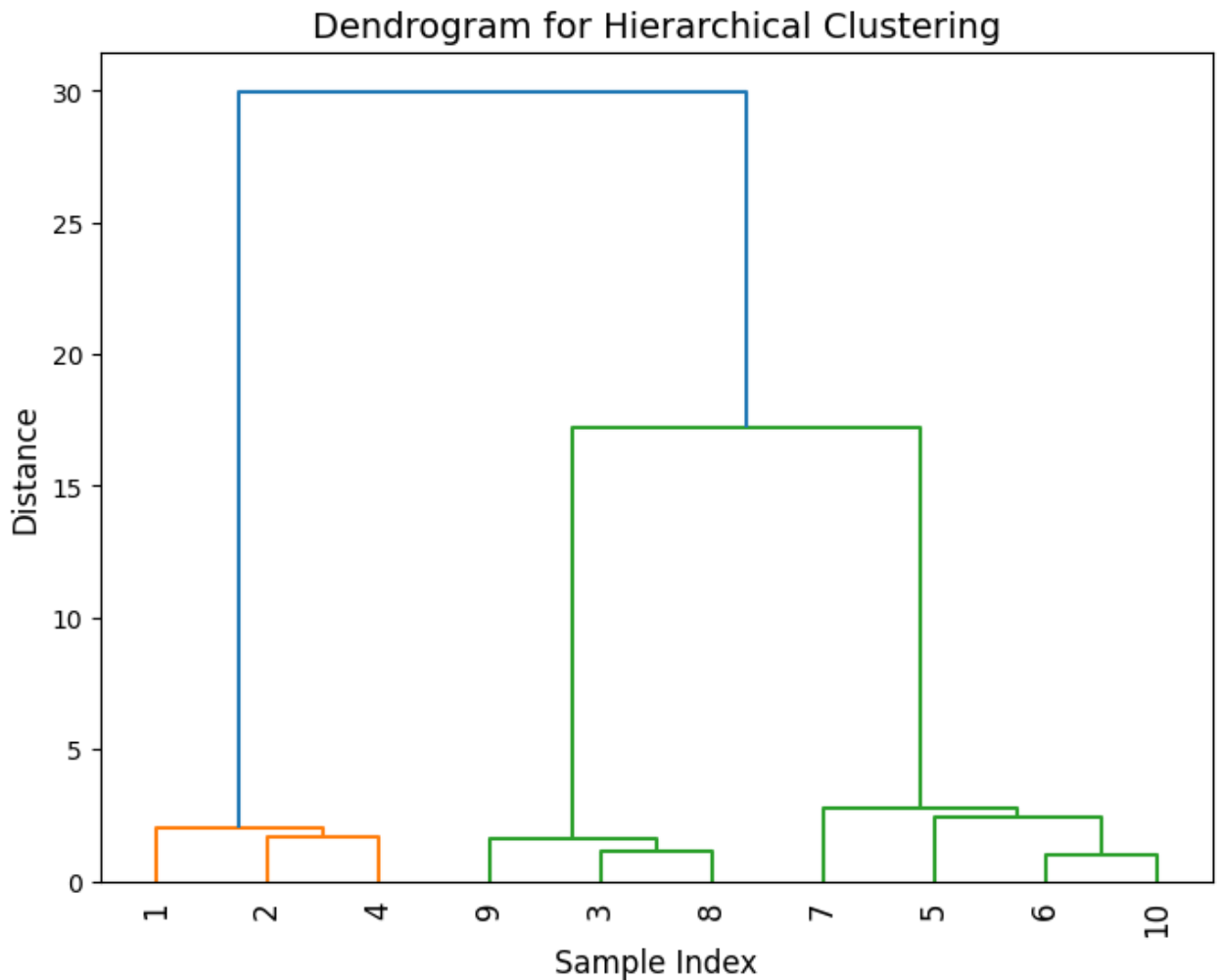
# Create a small dataset
np.random.seed(42)
data, _ = make_blobs(n_samples=10, centers=3, cluster_std=1, random_s
df = pd.DataFrame(data, columns=['Feature1', 'Feature2'])

# Perform hierarchical clustering (Agglomerative)
```



```
linkage_matrix = linkage(data, method='ward') # 'ward' minimizes var:

# Plot the dendrogram
plt.figure(figsize=(8, 6))
dendrogram(linkage_matrix, labels=range(1, len(data) + 1), leaf_rotat:
plt.title("Dendrogram for Hierarchical Clustering", fontsize=14)
plt.xlabel("Sample Index", fontsize=12)
plt.ylabel("Distance", fontsize=12)
plt.show()
```



Explanation of the Example

1. **Dataset:** A small synthetic dataset with 10 data points and two features is used for simplicity.
2. **Clustering Method:** Agglomerative clustering is performed using the **Ward** linkage, which minimizes the variance within clusters.
3. **Dendrogram:** The dendrogram visualizes the clustering process, showing at what distance clusters were merged.

Output

The dendrogram graphically displays how clusters are formed at different levels of the hierarchy. You can cut the dendrogram at a desired height to determine the final number of clusters.

✓ 4. Distribution-based Clustering

Using distribution-based clustering, data points are generated and organized according to their propensity to fall into the same probability distribution (such as a Gaussian, binomial, or other) within the data. The data elements are grouped using a probability-based distribution that is based on statistical distributions. Included are data objects that have a higher likelihood of being in the cluster. **A data point is less likely to be included in a cluster the further it is from the cluster's central point,** which exists in every cluster.

A notable drawback of density and boundary-based approaches is the need to specify the clusters a priori for some algorithms, and primarily the definition of the cluster form for the bulk of algorithms. There must be at least **one tuning or hyper-parameter selected**, and while doing so should be simple, getting it wrong could have unanticipated repercussions. Distribution-based clustering has a definite advantage over proximity and centroid-based clustering approaches in terms of flexibility, accuracy, and cluster structure. The key issue is that, in order to avoid overfitting, many clustering methods only work with simulated or manufactured data, or when the bulk of the data points certainly belong to a preset distribution. **The most popular distribution-based clustering algorithm is Gaussian Mixture Model.**

Distribution-based clustering assumes that data points are generated from a mixture of probability distributions (e.g., Gaussian distributions). It identifies clusters by estimating the underlying statistical distribution of the data. A common method is **Gaussian Mixture Models (GMM)**, which uses Expectation-Maximization (EM) to assign probabilities of cluster membership.

✓ Advantages

- **Probabilistic assignment:** Assigns probabilities of belonging to each cluster, allowing for soft clustering.
- **Flexibility in cluster shape:** Can model elliptical or irregularly shaped clusters.
- **Handles overlap:** Suitable for datasets where clusters overlap.

Disadvantages

- **Model complexity:** Requires careful selection of the distribution type and number of clusters.
- **Sensitive to initialization:** Poor initialization can lead to suboptimal results.
- **Scalability:** Computationally expensive for large datasets.

```
### **Practical Exercise: Gaussian Mixture Model (GMM)**

# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture
from sklearn.datasets import make_blobs

# Generate synthetic data
np.random.seed(42)
data, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)
df = pd.DataFrame(data, columns=['Feature1', 'Feature2'])

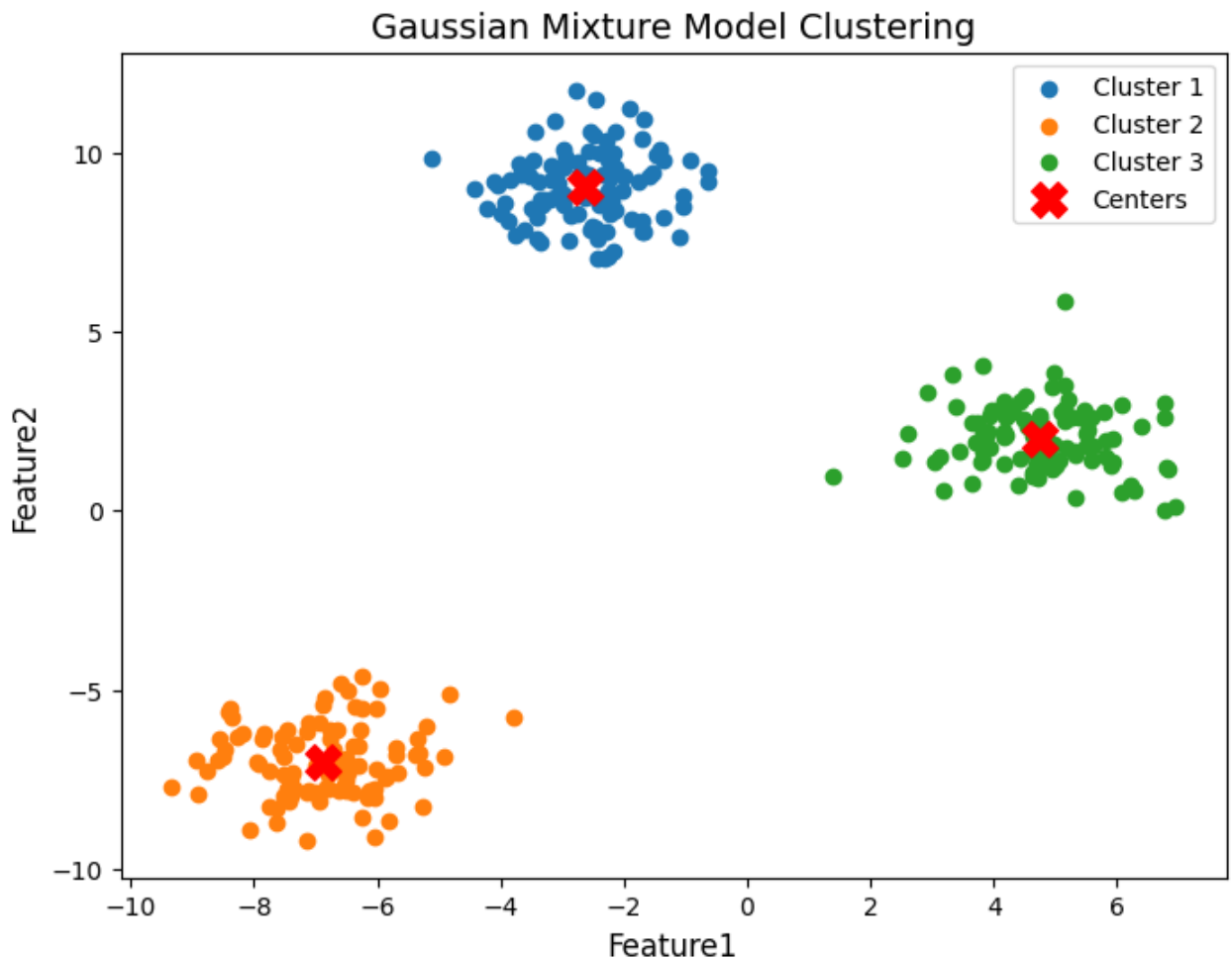
# Fit Gaussian Mixture Model
gmm = GaussianMixture(n_components=3, random_state=42)
df['Cluster'] = gmm.fit_predict(data)

# Visualize the clusters
plt.figure(figsize=(8, 6))
```

```
for cluster in range(3):
    cluster_data = df[df['Cluster'] == cluster]
    plt.scatter(cluster_data['Feature1'], cluster_data['Feature2'], label=cluster)

# Overlay cluster centers
centers = gmm.means_
plt.scatter(centers[:, 0], centers[:, 1], s=200, c='red', marker='X', label='Centers')

# Customize the plot
plt.title('Gaussian Mixture Model Clustering', fontsize=14)
plt.xlabel('Feature1', fontsize=12)
plt.ylabel('Feature2', fontsize=12)
plt.legend()
plt.show()
```



Explanation of the Example

1. **Dataset:** A synthetic dataset with three Gaussian-distributed clusters is created using `make_blobs`.
2. **Model:** A Gaussian Mixture Model (GMM) with three components is fitted to the data.
3. **Visualization:** Data points are plotted with distinct colors for each cluster, and cluster centers are marked.

Output

The plot displays soft clusters, where points are probabilistically assigned to clusters. The GMM identifies overlapping and non-spherical clusters effectively.

Applications of Clustering

- **Marketing:** Clustering helps identify and categorize customer segments for targeted marketing strategies.
- **Biology:** It aids in classifying plant and animal species based on shared characteristics.
- **Libraries:** Books can be grouped by topics and content for efficient organization.
- **Insurance:** Clustering assists in analyzing customer policies, understanding customer behavior, and detecting fraudulent activities.
- **City Planning:** Used to group houses and analyze their values based on location and other factors.
- **Earthquake Studies:** Identifies high-risk zones by analyzing earthquake-affected areas.
- **Image Processing:** Groups similar images, classifies them by content, and detects patterns in image datasets.
- **Genetics:** Helps group genes with similar expression patterns, revealing biological networks and processes.
- **Finance:** Used to segment markets based on customer behavior, analyze stock market trends, and assess investment risks.
- **Customer Service:** Groups customer inquiries and complaints to identify

recurring issues and develop targeted solutions.

- **Manufacturing:** Clusters similar products, optimizes production workflows, and identifies manufacturing defects.
- **Medical Diagnosis:** Groups patients with similar symptoms or diseases to improve diagnoses and treatment plans.
- **Fraud Detection:** Detects unusual patterns in financial transactions to identify potential fraud or financial crimes.
- **Traffic Analysis:** Analyzes traffic data patterns, such as peak times and routes, to enhance transportation planning.
- **Social Network Analysis:** Identifies groups or communities within networks, shedding light on social behavior, influence, and trends.
- **Cybersecurity:** Clusters network traffic or system behaviors to detect and prevent potential cyberattacks.
- **Climate Analysis:** Groups climate data patterns like temperature and rainfall to study climate change and its impacts.
- **Sports Analysis:** Groups player or team performance data to evaluate strengths, weaknesses, and strategic opportunities.
- **Crime Analysis:** Clusters crime data by location, time, and type to identify hotspots, predict trends, and improve prevention strategies.

✓ Dimensionality Reduction

Dimensionality reduction is the process of reducing the number of features in a dataset while preserving as much information as possible. This technique is useful for improving the performance of machine learning algorithms and for data visualization. Examples of dimensionality reduction algorithms include

Dimensionality reduction is the process of reducing the number of features in a dataset while preserving as much information as possible.

- Principal Component Analysis (**PCA**): Linear Transformation for Reduced Dimensions
- Linear Discriminant Analysis (**LDA**): Dimensionality Reduction for Discrimination
- Non-negative Matrix Factorization (**NMF**): Decomposing Data into Non-negative Components
- Locally Linear Embedding (**LLE**): Preserving Local Geometry in Reduced Dimensions
- **Isomap**: Capturing Global Relationships in Reduced Dimensions
- t-Distributed Stochastic Neighbor Embedding(**t-SNE**) is a dimensionality reduction.

Principal Component Analysis(PCA)

As the number of features or dimensions in a dataset increases, the amount of data required to obtain a statistically significant result increases exponentially. This can lead to issues such as overfitting, increased computation time, and reduced accuracy of machine learning models this is known as the curse of dimensionality problems that arise while working with high-dimensional data.

As the number of dimensions increases, the number of possible combinations of features increases exponentially, which makes it computationally difficult to obtain a representative sample of the data. It becomes expensive to perform tasks such as clustering or classification because the algorithms need to process a much larger feature space, which increases computation time and complexity.

✓ Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a dimensionality reduction technique used to simplify large datasets while retaining the most important information. It identifies new axes (principal components) that capture the maximum variance in the data and transforms the data into a lower-dimensional space.

Advantages

- **Reduces complexity:** Simplifies high-dimensional datasets, making them easier to visualize and analyze.
- **Improves efficiency:** Reduces computational requirements for machine learning models.
- **De-noises data:** Removes less significant features, reducing the impact of noise.

Disadvantages

- **Interpretability:** The transformed components are linear combinations of original features and may not be intuitive.
- **Loss of information:** Some variance is inevitably lost when reducing dimensions.
- **Assumes linearity:** PCA assumes linear relationships, which may not always hold in complex datasets.

```
### **Practical Exercise: PCA for Dimensionality Reduction**
```

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# Load the Iris dataset
iris = load_iris()
data = iris.data
feature_names = iris.feature_names
```

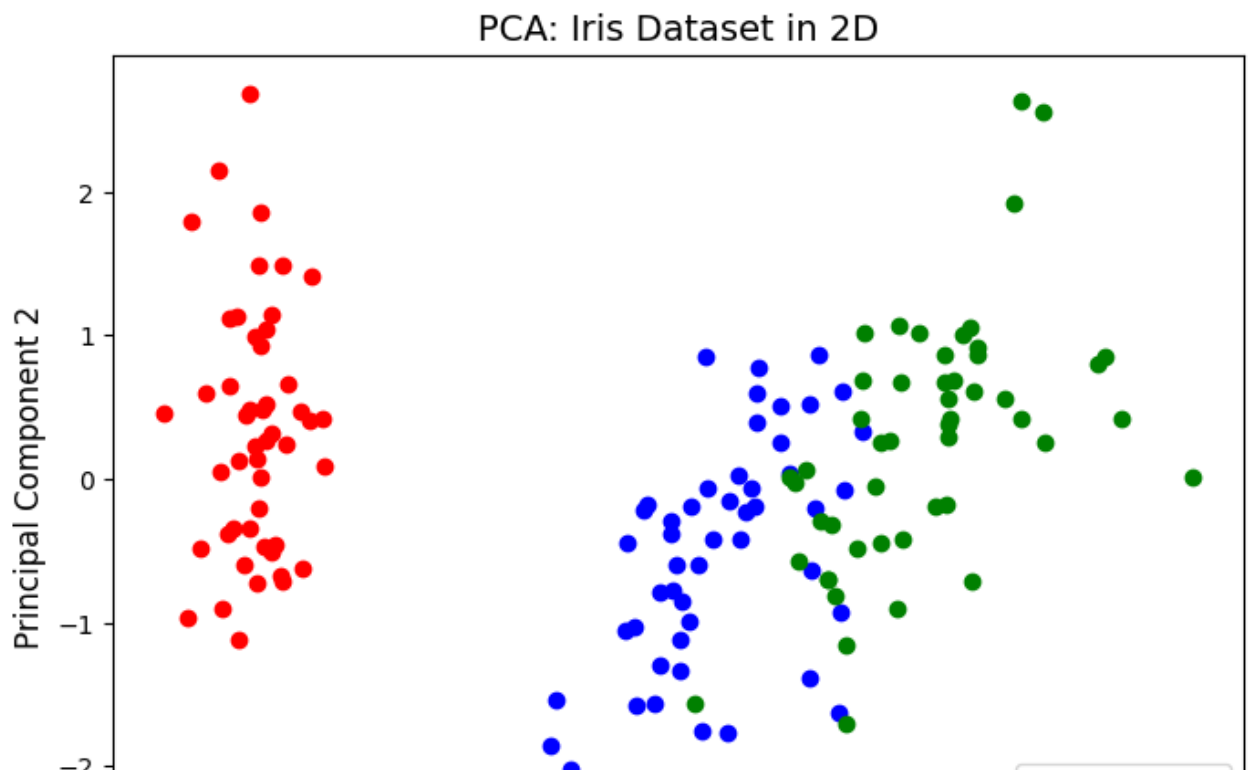
```
# Standardize the data
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)

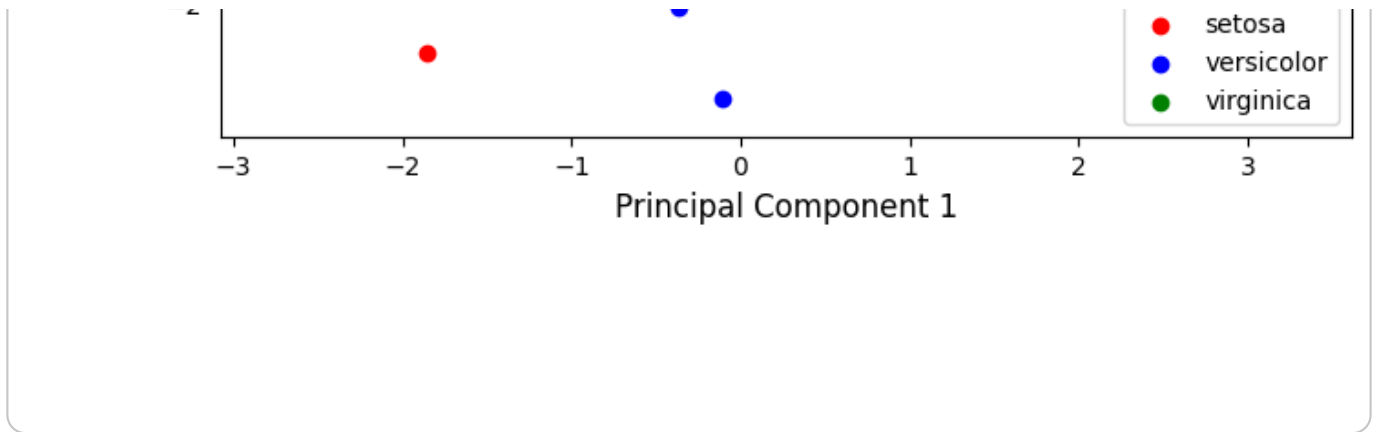
# Apply PCA to reduce to 2 dimensions
pca = PCA(n_components=2)
data_pca = pca.fit_transform(data_scaled)

# Create a DataFrame for visualization
pca_df = pd.DataFrame(data_pca, columns=['Principal Component 1', 'Principal Component 2'])
pca_df['Target'] = iris.target

# Visualize the data in reduced dimensions
plt.figure(figsize=(8, 6))
for target, color in zip(np.unique(iris.target), ['red', 'blue', 'green']):
    subset = pca_df[pca_df['Target'] == target]
    plt.scatter(subset['Principal Component 1'], subset['Principal Component 2'], color=color)

# Customize the plot
plt.title('PCA: Iris Dataset in 2D', fontsize=14)
plt.xlabel('Principal Component 1', fontsize=12)
plt.ylabel('Principal Component 2', fontsize=12)
plt.legend()
plt.show()
```





Explanation of the Example

1. **Dataset:** The Iris dataset, which has 4 features, is reduced to 2 principal components.
2. **Standardization:** Data is standardized to ensure that PCA captures the relative importance of features.
3. **PCA Transformation:** The data is transformed into two principal components.
4. **Visualization:** The reduced dataset is plotted in a 2D space, showing the separation between different species.

Output

The scatterplot displays data points in terms of the two principal components, effectively reducing four-dimensional data to two while preserving the key patterns in the dataset.

This exercise demonstrates PCA's power in simplifying high-dimensional datasets while retaining essential information.

```
import pandas as pd
import numpy as np

# Here we are using inbuilt dataset of scikit learn
from sklearn.datasets import load_breast_cancer

# instantiating
cancer = load_breast_cancer(as_frame=True)
# creating dataframe
df = cancer.frame

# checking shape
print('Original Dataframe shape :',df.shape)

# Input features
X = df[cancer['feature_names']]
print('Inputs Dataframe shape    :', X.shape)
```

```
Original Dataframe shape : (569, 31)
Inputs Dataframe shape   : (569, 30)
```

```
# Mean
X_mean = X.mean()

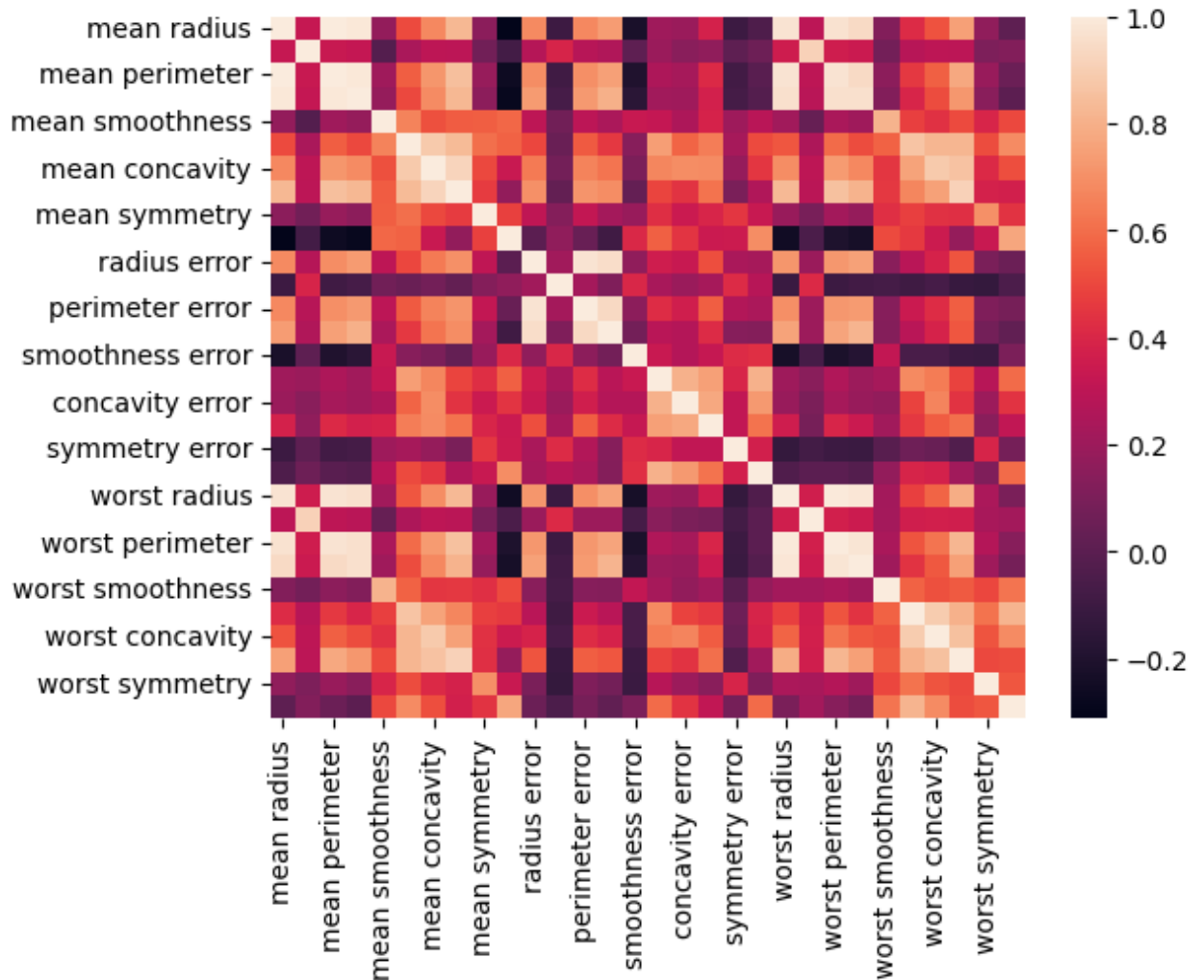
# Standard deviation
X_std = X.std()

# Standardization
Z = (X - X_mean) / X_std
```

The **covariance matrix** helps us visualize how strong the dependency of two features is with each other in the feature space.

```
# covariance
c = Z.cov()

# Plot the covariance matrix
import matplotlib.pyplot as plt
import seaborn as sns
sns.heatmap(c)
plt.show()
```



```
eigenvalues, eigenvectors = np.linalg.eig(c)
print('Eigen values:\n', eigenvalues)
print('Eigen values Shape:', eigenvalues.shape)
print('Eigen Vector Shape:', eigenvectors.shape)
```

```
Eigen values:
[1.32816077e+01 5.69135461e+00 2.81794898e+00 1.98064047e+00
 1.64873055e+00 1.20735661e+00 6.75220114e-01 4.76617140e-01
 4.16894812e-01 3.50693457e-01 2.93915696e-01 2.61161370e-01
 2.41357496e-01 1.57009724e-01 9.41349650e-02 7.98628010e-02
 5.93990378e-02 5.26187835e-02 4.94775918e-02 1.33044823e-04
 7.48803097e-04 1.58933787e-03 6.90046388e-03 8.17763986e-03
 1.54812714e-02 1.80550070e-02 2.43408378e-02 2.74394025e-02
 3.11594025e-02 2.99728939e-02]
Eigen values Shape: (30,)
Eigen Vector Shape: (30, 30)
```

```
# Index the eigenvalues in descending order
idx = eigenvalues.argsort()[::-1]

# Sort the eigenvalues in descending order
eigenvalues = eigenvalues[idx]

# sort the corresponding eigenvectors accordingly
eigenvectors = eigenvectors[:,idx]
```

```
explained_var = np.cumsum(eigenvalues) / np.sum(eigenvalues)
explained_var
```

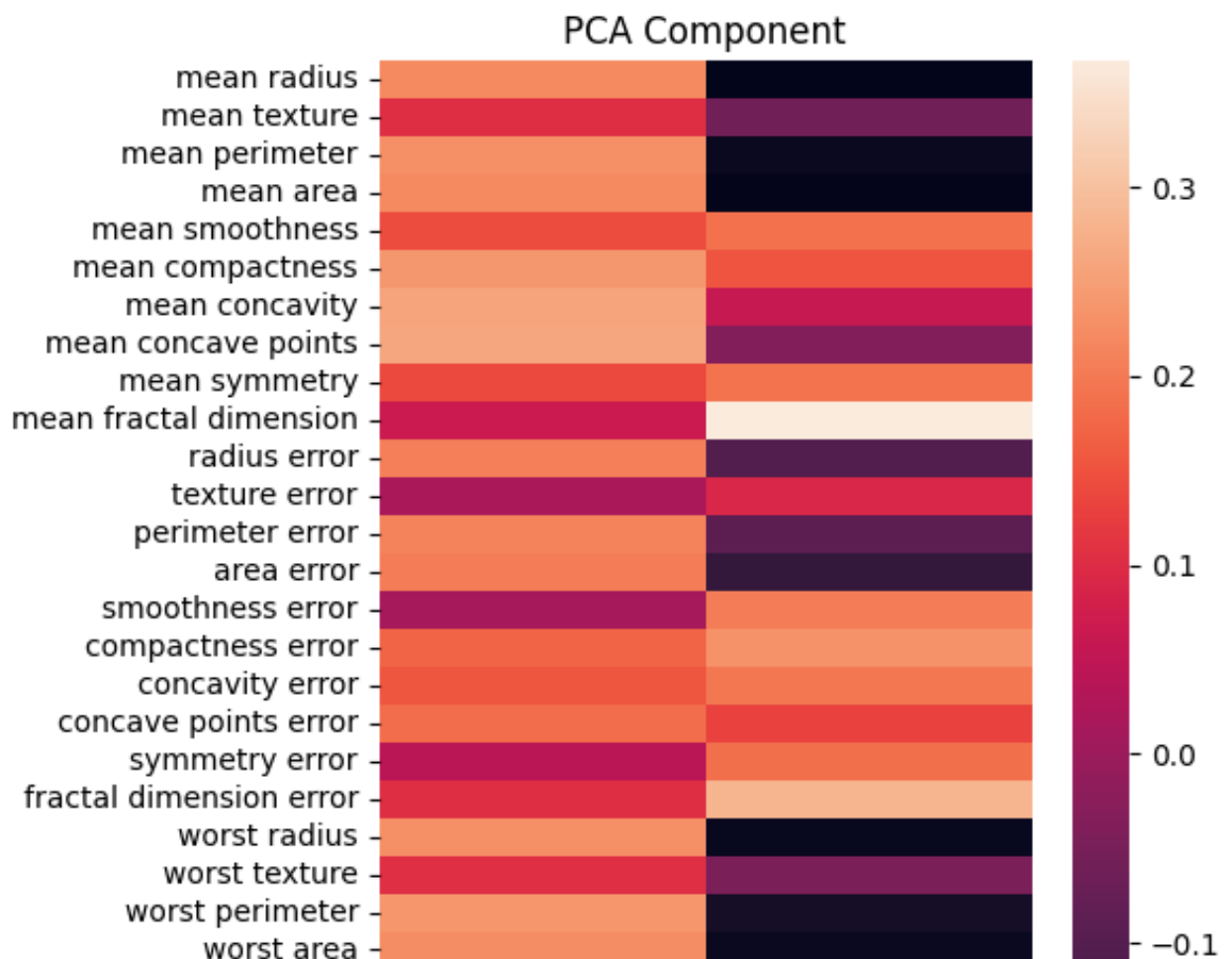
```
array([0.44272026, 0.63243208, 0.72636371, 0.79238506, 0.84734274,
        0.88758796, 0.9100953 , 0.92598254, 0.93987903, 0.95156881,
        0.961366 , 0.97007138, 0.97811663, 0.98335029, 0.98648812,
        0.98915022, 0.99113018, 0.99288414, 0.9945334 , 0.99557204,
        0.99657114, 0.99748579, 0.99829715, 0.99889898, 0.99941502,
        0.99968761, 0.99991763, 0.99997061, 0.99999557, 1.        ])
```

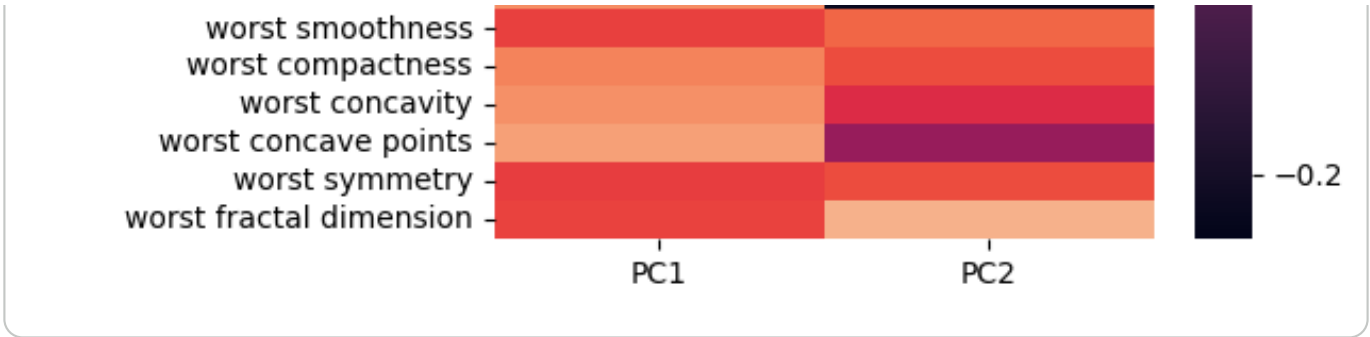
```
n_components = np.argmax(explained_var >= 0.50) + 1
n_components
```

2

```
# PCA component or unit matrix
u = eigenvectors[:, :n_components]
pca_component = pd.DataFrame(u,
                             index = cancer['feature_names'],
                             columns = ['PC1', 'PC2']
                             )
```

```
# plotting heatmap
plt.figure(figsize =(5, 7))
sns.heatmap(pca_component)
plt.title('PCA Component')
plt.show()
```





✓ Linear Discriminant Analysis in Machine Learning

As we know that while dealing with a high dimensional dataset then we must apply some dimensionality reduction techniques to the data at hand so, that we can explore the data and utilize it for modeling in an efficient manner. In this article, we will learn about one such dimensionality reduction technique that is used to map high dimensional data to a comparatively lower dimension without much data loss.

Linear Discriminant Analysis (LDA) is a supervised machine learning technique used for dimensionality reduction and classification. LDA seeks to maximize the separation between multiple classes by finding a linear combination of features that best separates the classes. It projects the data onto a lower-dimensional space while preserving class discriminability.

Advantages

- **Effective for classification:** Enhances class separation in reduced dimensions.
- **Noise reduction:** Reduces irrelevant features, improving model performance.
- **Low computational cost:** Simpler and faster compared to some non-linear methods.

Disadvantages

- **Linearity assumption:** Assumes data is linearly separable, which might not hold in complex datasets.
- **Requires labeled data:** Unlike PCA, LDA requires class labels for dimensionality reduction.
- **Sensitivity to outliers:** Can be affected by outliers, impacting performance.

**Practical Exercise: LDA for Dimensionality Reduction and Visual:


```
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.datasets import load_iris

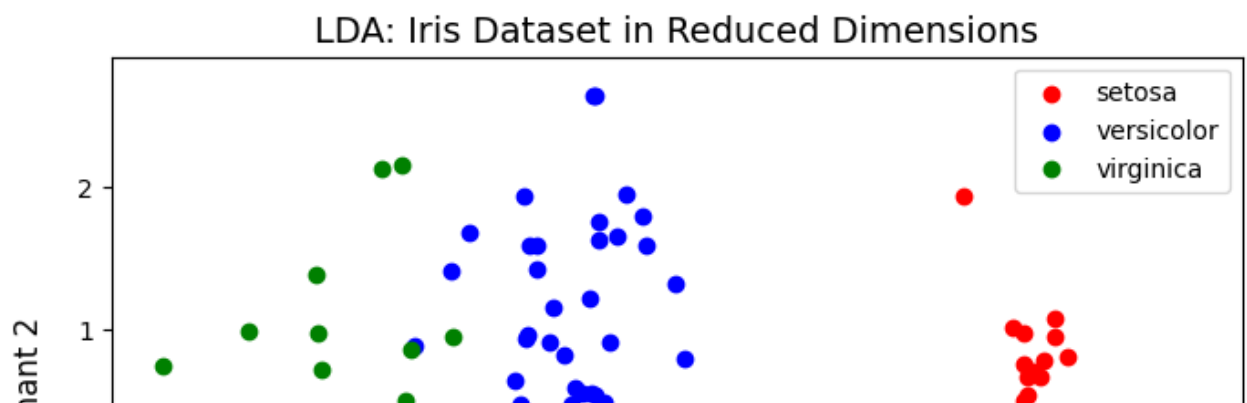
# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
class_names = iris.target_names

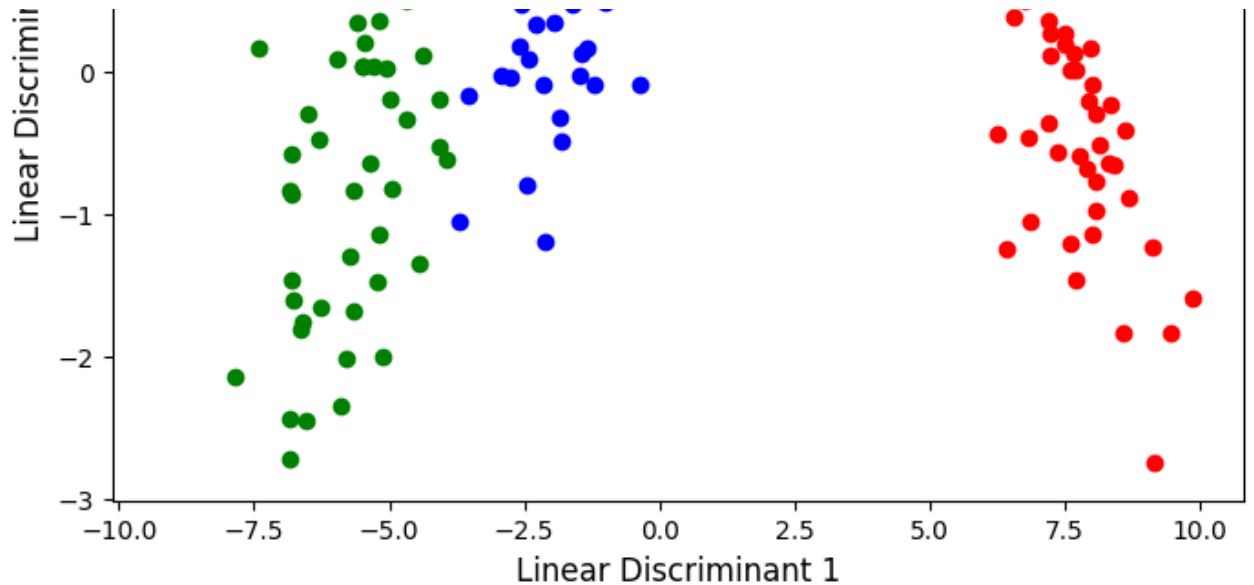
# Apply LDA to reduce dimensions
lda = LinearDiscriminantAnalysis(n_components=2)
X_lda = lda.fit_transform(X, y)

# Create a DataFrame for visualization
lda_df = pd.DataFrame(X_lda, columns=['LD1', 'LD2'])
lda_df['Target'] = y

# Visualize the data
plt.figure(figsize=(8, 6))
for target, color in zip(np.unique(y), ['red', 'blue', 'green']):
    subset = lda_df[lda_df['Target'] == target]
    plt.scatter(subset['LD1'], subset['LD2'], label=class_names[target])

# Customize the plot
plt.title('LDA: Iris Dataset in Reduced Dimensions', fontsize=14)
plt.xlabel('Linear Discriminant 1', fontsize=12)
plt.ylabel('Linear Discriminant 2', fontsize=12)
plt.legend()
plt.show()
```





Explanation of the Example

1. **Dataset:** The Iris dataset with 4 features and 3 labeled classes is used.
2. **LDA Application:** LDA reduces the dataset to 2 dimensions while maximizing the separation between the 3 classes.
3. **Visualization:** Data is plotted in terms of the two linear discriminants, highlighting class separability.

✓ Non-Negative Matrix Factorization

Nonnegative Matrix Factorization is a matrix factorization method where we constrain the matrices to be nonnegative. In order to understand NMF, we should clarify the underlying intuition between matrix factorization.

For a matrix A of dimensions $m \times n$, where each element is ≥ 0 , NMF can factorize it into two matrices W and H having dimensions $m \times k$ and $k \times n$ respectively and these two matrices only contain non-negative elements. Here, matrix A is defined as:

$$A_{\{m \times n\}} = W_{\{m \times k\}} H_{\{k \times n\}}$$

where,

$A \rightarrow$ Original Input Matrix (Linear combination of W & H)

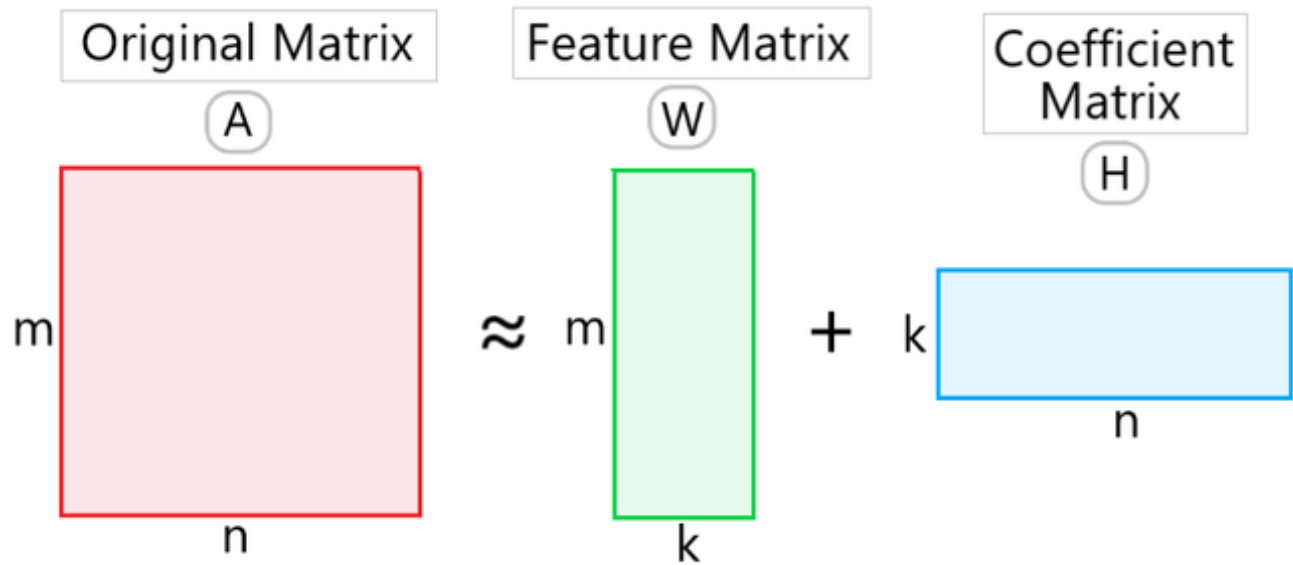
$W \rightarrow$ Feature Matrix

$H \rightarrow$ Coefficient Matrix (Weights associated with W)

$k \rightarrow$ Low rank approximation of A ($k \leq \min(m, n)$)

This method is widely used in performing tasks such as feature reduction in Facial Recognition and for various NLP tasks.

Intuition:



<https://dataloop.ai/wp-content/uploads/2022/04/non-negative-matrix-factorization.png>

The objective of NMF is dimensionality reduction and feature extraction. So, when we set lower dimension as k , the goal of NMF is to find two matrices $W \in \mathbb{R}^{m \times k}$ and $H \in \mathbb{R}^{n \times k}$ having only nonnegative elements. (As shown in Fig 1)

Therefore, by using NMF we are able to obtain factorized matrices having significantly lower dimensions than those of the product matrix. Intuitively, NMF assumes that the original input is made of a set of hidden features, represented by each column of W matrix and each column in H matrix represents the 'coordinates of a data point' in the matrix W . In simple terms, it contains the weights associated with matrix W .

In this, each data point that is represented as a column in A , can be approximated by an additive combination of the non-negative vectors, which are represented as columns in W .

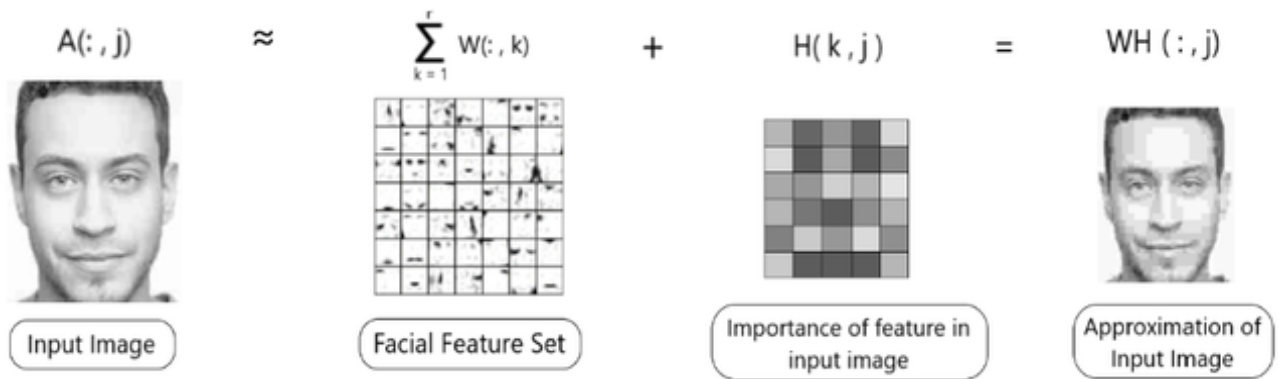
How Does it Work?

- NMF decomposes multivariate data by creating a user-defined number of features. Each feature is a linear combination of the original attribute set; the coefficients of these linear combinations are non-negative.
- NMF decomposes a data matrix V into the product of two lower rank matrices W and H so that V is approximately equal to W times H .
- NMF uses an iterative procedure to modify the initial values of W and H so that the product approaches V . The procedure terminates when the approximation error converges or the specified number of iterations is reached
- During model apply, an NMF model maps the original data into the new set of attributes (features) discovered by the model.

****Real-life example: ****

Let us consider some real-life examples to understand the working of the NMF algorithm. Let's take a case of image processing.

Suppose, we have an input image, having pixels that form matrix A . Using NMF, we factorize it into two matrices, one containing the facial feature set [**Matrix W**] and the other containing the importance of each facial feature in the input image, i.e. the weights [**Matrix H**]. (As shown in Fig 2.)



<https://images.javatpoint.com/tutorial/machine-learning/images/nonnegative-matrix-factorization2.png>

NMF is used in major applications such as image processing, text mining, spectral data analysis and many more. Currently, there is an ongoing research on NMF to increase its efficiency and robustness. Other research is being done on collective factorization, efficient update of matrices etc. as well. For any doubt/query, comment below.

Non-Negative Matrix Factorization (NMF) is a dimensionality reduction and feature extraction technique that factors a non-negative data matrix (V) into two lower-dimensional non-negative matrices (W) and (H), such that $V \approx W \cdot H$. This method is commonly used for datasets like images or text, where non-negativity makes sense (e.g., pixel intensities or word counts).

✓ Advantages

- **Non-negativity constraint:** Ensures interpretability of the factors (e.g., additive parts in images or topics in text).
- **Sparsity:** Encourages sparsity in the factor matrices, leading to compact representations.
- **Good for high-dimensional data:** Works well for feature extraction from large datasets.

Disadvantages

- **Sensitive to initialization:** Requires careful initialization to ensure convergence to a meaningful solution.
- **Loss of information:** Approximates the original data, so some information may be lost.
- **Non-convex optimization:** The algorithm may converge to local minima.

```
# Import libraries
import numpy as np
import pandas as pd
from sklearn.decomposition import NMF
from sklearn.feature_extraction.text import TfidfVectorizer

# Sample text dataset
documents = [
    "The cat sat on the mat",
    "The dog barked at the mailman",
    "Cats and dogs are common pets",
    "The mat was made of wool",
    "Mailmen often bring packages"
]

# Convert text data into a TF-IDF matrix
```

```
vectorizer = TfidfVectorizer(stop_words='english')
tfidf_matrix = vectorizer.fit_transform(documents)

# Apply NMF with 2 components (topics)
nmf = NMF(n_components=2, random_state=42)
W = nmf.fit_transform(tfidf_matrix) # Document-topic matrix
H = nmf.components_                # Topic-term matrix

# Display results
terms = vectorizer.get_feature_names_out()
topics = pd.DataFrame(H, columns=terms)
topics.index = [f'Topic {i+1}' for i in range(H.shape[0])]

print("Top words in each topic:")
print(topics)

# Document-topic representation
document_topics = pd.DataFrame(W, columns=[f'Topic {i+1}' for i in range(W.shape[1])])
document_topics['Document'] = documents
print("\nDocument-topic matrix:")
print(document_topics)
```

```
Top words in each topic:
      barked  bring  cat  cats  common  dog
Topic 1  0.000000  0.000004  0.475034  0.000073  0.000073  0.000000  0
Topic 2  0.494317  0.097414  0.000000  0.094291  0.094291  0.494317  0

      mailman  mailmen  mat  packages  pets  sat
Topic 1  0.000000  0.000004  0.868904  0.000004  0.000073  0.475034  0
Topic 2  0.494317  0.097414  0.000000  0.097414  0.094291  0.000000  0

Document-topic matrix:
      Topic 1  Topic 2  Document
0  0.646468  0.000000  The cat sat on the mat
1  0.000000  1.074150  The dog barked at the mailman
2  0.000129  0.236591  Cats and dogs are common pets
3  0.646468  0.000000  The mat was made of wool
4  0.000012  0.211679  Mailmen often bring packages
```


Explanation of the Example

1. **Dataset:** A small collection of text documents is used as input.
2. **TF-IDF Transformation:** Text data is transformed into a numerical matrix using TF-IDF to capture term importance.
3. **NMF Application:** NMF decomposes the TF-IDF matrix into two smaller matrices:
 - **Document-topic matrix ((W)):** Represents the distribution of topics in each document.
 - **Topic-term matrix ((H)):** Contains the importance of each word for each topic.
4. **Interpretation:** The top words in each topic and the topic distribution for each document are displayed.

✓ Locally Linear Embedding in machine learning

Locally Linear Embedding (LLE) is a non-linear dimensionality reduction technique. It maps high-dimensional data to a lower-dimensional space while preserving the local relationships among data points. LLE assumes that data lies on a low-dimensional manifold embedded in a high-dimensional space, and it approximates each data point using a linear combination of its nearest neighbors.

Advantages

- **Captures non-linear structures:** Preserves non-linear relationships in the data.
- **No assumptions about global geometry:** Focuses on local relationships, making it robust for manifold learning.
- **Unsupervised:** Does not require labeled data.

Disadvantages

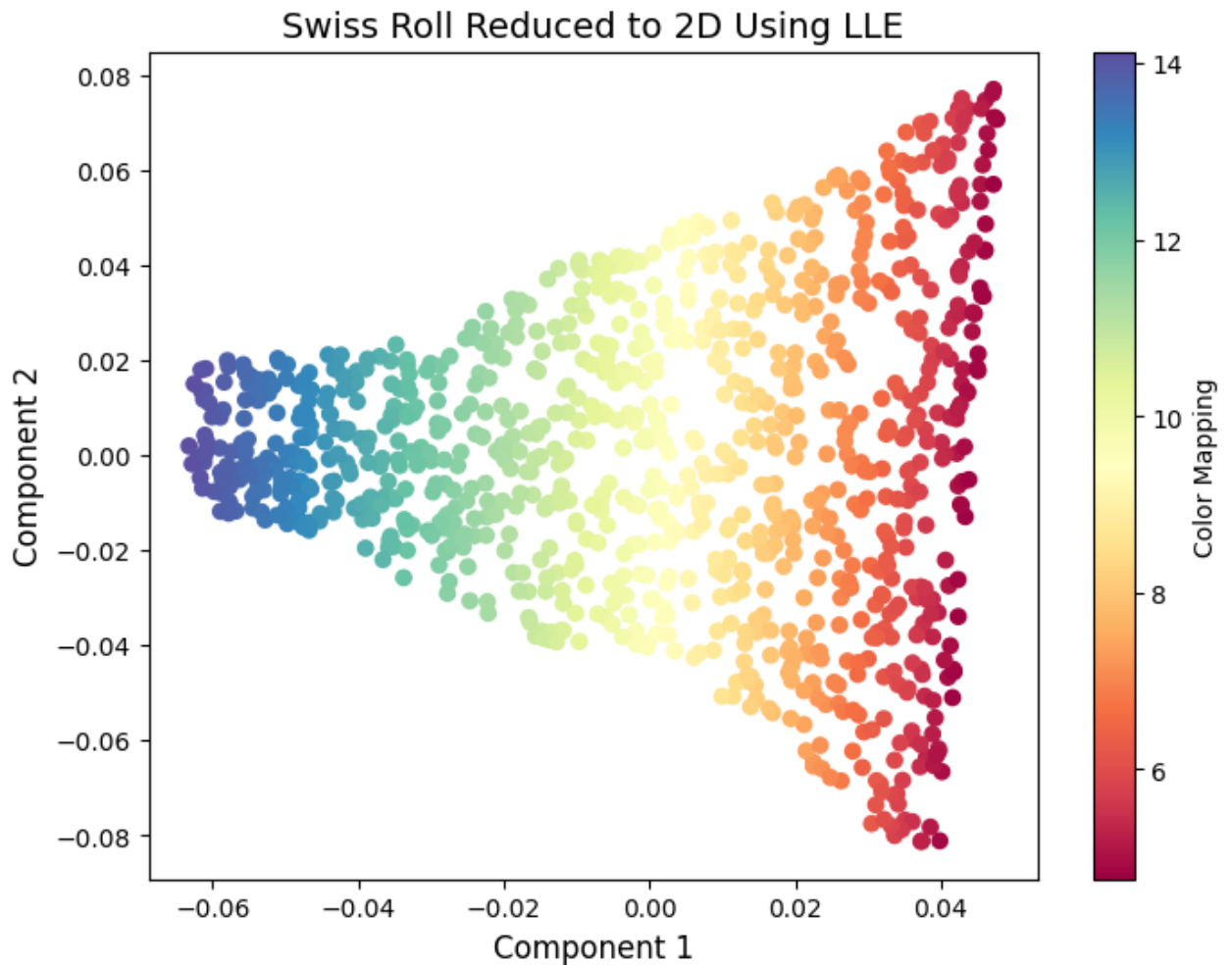
- **Sensitive to noise:** Small perturbations in the data can affect results.
- **Computationally expensive:** Involves finding nearest neighbors and solving large eigenvalue problems.
- **Requires parameter tuning:** The number of neighbors ((k)) significantly impacts performance.

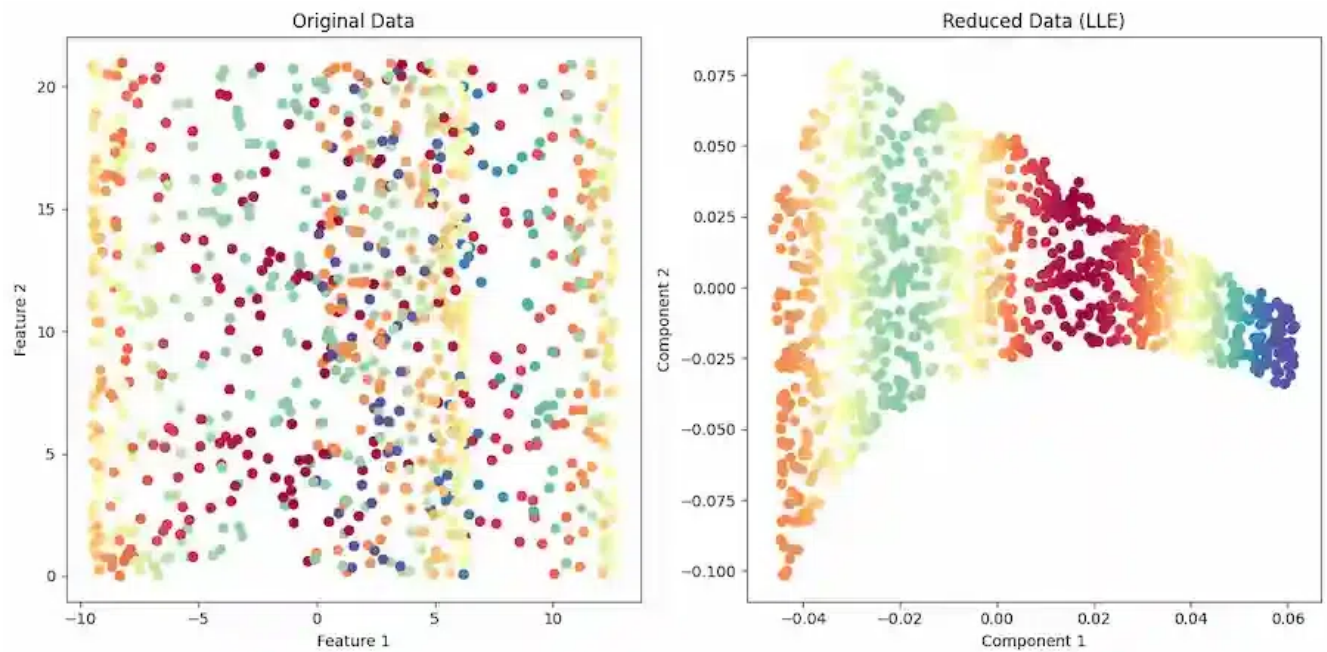
```
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_swiss_roll
from sklearn.manifold import LocallyLinearEmbedding

# Generate a 3D Swiss roll dataset
np.random.seed(42)
data, color = make_swiss_roll(n_samples=1000, noise=0.1)

# Apply Locally Linear Embedding (reduce to 2D)
lle = LocallyLinearEmbedding(n_neighbors=12, n_components=2, random_s
data_lle = lle.fit_transform(data)
```

```
# Plot the results
plt.figure(figsize=(8, 6))
plt.scatter(data_lle[:, 0], data_lle[:, 1], c=color, cmap=plt.cm.Spectral)
plt.title("Swiss Roll Reduced to 2D Using LLE", fontsize=14)
plt.xlabel("Component 1", fontsize=12)
plt.ylabel("Component 2", fontsize=12)
plt.colorbar(label="Color Mapping")
plt.show()
```





✎ Explanation of the Example

1. **Dataset:** A 3D Swiss roll dataset (a non-linear manifold) is created.
2. **LLE Application:** The algorithm reduces the 3D data to 2D by preserving local relationships using 12 nearest neighbors.
3. **Visualization:** The reduced data is plotted, showing how LLE unfolds the Swiss roll while retaining its structure.

Output

The scatterplot demonstrates how LLE successfully reduces the dimensionality of the Swiss roll while preserving its local geometry, making it easier to visualize the underlying structure.

Advantages of LLE The dimensionality reduction method known as locally linear embedding (LLE) has many benefits for data processing and visualization. The following are LLE's main benefits:

- **Preservation of Local Structures:** LLE is excellent at maintaining the in-data local relationships or structures. It successfully captures the inherent geometry of nonlinear manifolds by maintaining pairwise distances between nearby data points.
- **Handling Non-Linearity:** LLE has the ability to capture nonlinear patterns and structures in the data, in contrast to linear techniques like Principal Component Analysis (PCA). When working with complicated, curved, or twisted datasets, it is especially helpful.
- **Dimensionality Reduction:** LLE lowers the dimensionality of the data while preserving its fundamental properties. Particularly when working with high-dimensional datasets, this reduction makes data presentation, exploration, and analysis simpler.

Disadvantages of LLE

- **Curse of Dimensionality:** LLE can experience the "curse of dimensionality" when used with extremely high-dimensional data, just like many other dimensionality reduction approaches. The number of neighbors required to capture local interactions rises as dimensionality does, potentially increasing the computational cost of the approach.
- **Memory and computational Requirements:** For big datasets, creating a weighted adjacency matrix as part of LLE might be memory-intensive. The eigenvalue decomposition stage can also be computationally taxing for big datasets.
- **Outliers and Noisy data:** LLE is susceptible to anomalies and jittery data points. The quality of the embedding may be affected and the local linear relationships may be distorted by outliers.

✓ Isomap: Capturing Global Relationships in Reduced Dimensions

Isomap (Isometric Mapping) is a non-linear dimensionality reduction technique that extends Multidimensional Scaling (MDS) by preserving the **geodesic distances** (shortest paths on a manifold) between data points. It is particularly useful for datasets with non-linear structures.

Advantages

- **Captures global structure:** Unlike Locally Linear Embedding (LLE), Isomap considers the global geometry of the data manifold.
- **Visualization-friendly:** Effectively reduces dimensions for visualization while maintaining meaningful relationships.
- **Unsupervised:** Requires no labeled data.

Disadvantages

- **Computationally expensive:** Finding the geodesic distances and solving eigenproblems is resource-intensive.
- **Sensitive to noise:** May struggle with noisy datasets or outliers.
- **Parameter dependency:** The number of neighbors (k) and connectivity can significantly impact results.

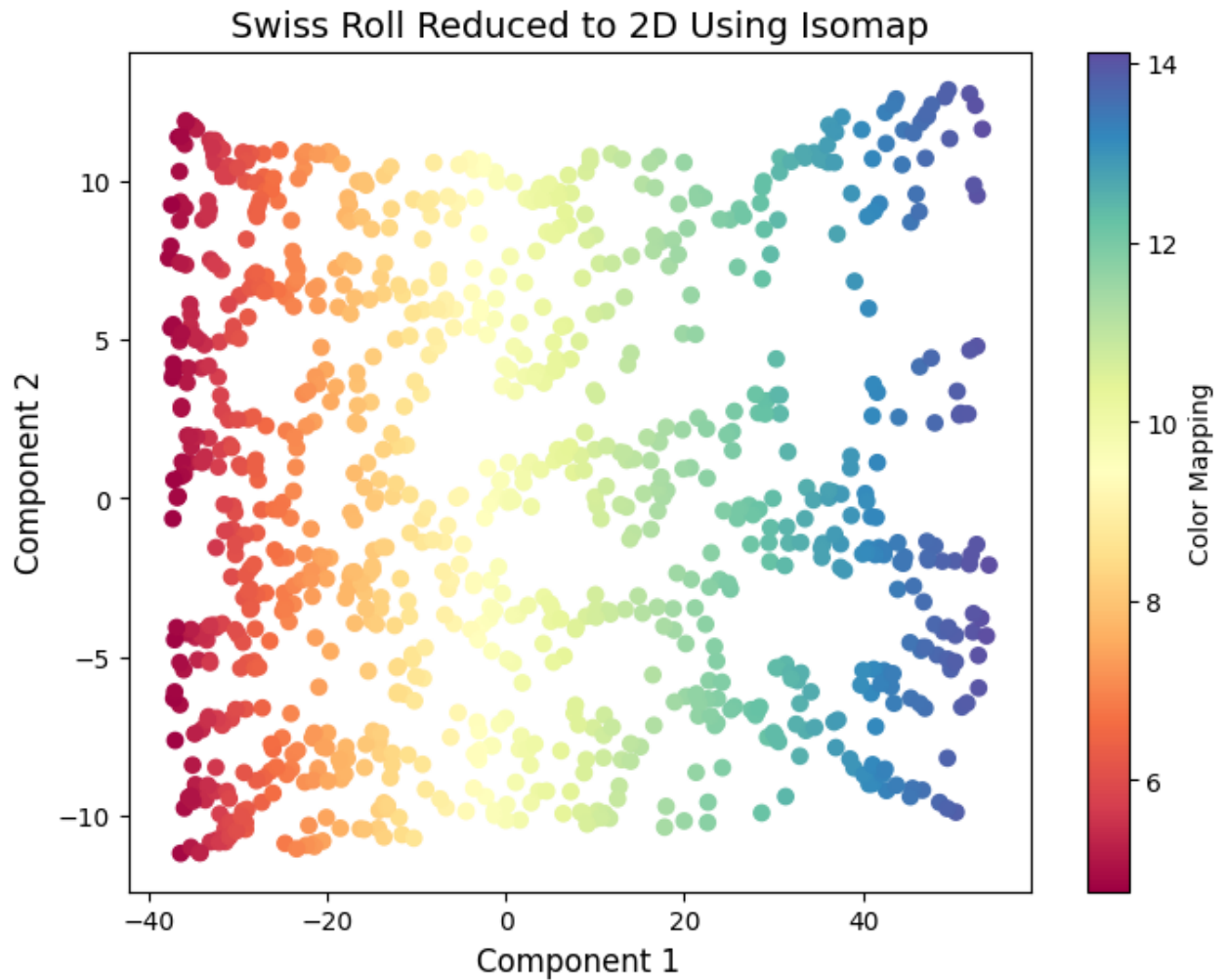
```
# Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_swiss_roll
from sklearn.manifold import Isomap

# Generate a 3D Swiss roll dataset
np.random.seed(42)
data, color = make_swiss_roll(n_samples=1000, noise=0.1)

# Apply Isomap (reduce to 2D)
isomap = Isomap(n_neighbors=10, n_components=2)
data_isomap = isomap.fit_transform(data)

# Plot the results
plt.figure(figsize=(8, 6))
plt.scatter(data_isomap[:, 0], data_isomap[:, 1], c=color, cmap=plt.cm
```

```
plt.title("Swiss Roll Reduced to 2D Using Isomap", fontsize=14)
plt.xlabel("Component 1", fontsize=12)
plt.ylabel("Component 2", fontsize=12)
plt.colorbar(label="Color Mapping")
plt.show()
```



Explanation of the Example

1. **Dataset:** A 3D Swiss roll dataset is generated to simulate a non-linear manifold.
2. **Isomap Application:** The algorithm reduces the data to 2D while preserving geodesic distances using 10 neighbors.
3. **Visualization:** The 2D scatterplot unfolds the Swiss roll, showing the global structure in the lower-dimensional space.

✓ t-SNE (t-Distributed Stochastic Neighbor Embedding)

t-SNE is a non-linear dimensionality reduction technique primarily used for visualizing high-dimensional data in 2D or 3D. It preserves the local structure of data by minimizing the divergence between probability distributions of points in high-dimensional and low-dimensional spaces.

Advantages

- **Great for visualization:** Effectively highlights clusters and patterns in complex datasets.
- **Preserves local structure:** Captures relationships among neighboring points in high-dimensional data.
- **Handles non-linear data:** Unfolds non-linear manifolds effectively.

Disadvantages

- **Computationally expensive:** Resource-intensive for large datasets.
- **Parameter-sensitive:** Requires tuning of perplexity and learning rate for optimal results.
- **Only for visualization:** Not suitable for tasks like dimensionality reduction for modeling.

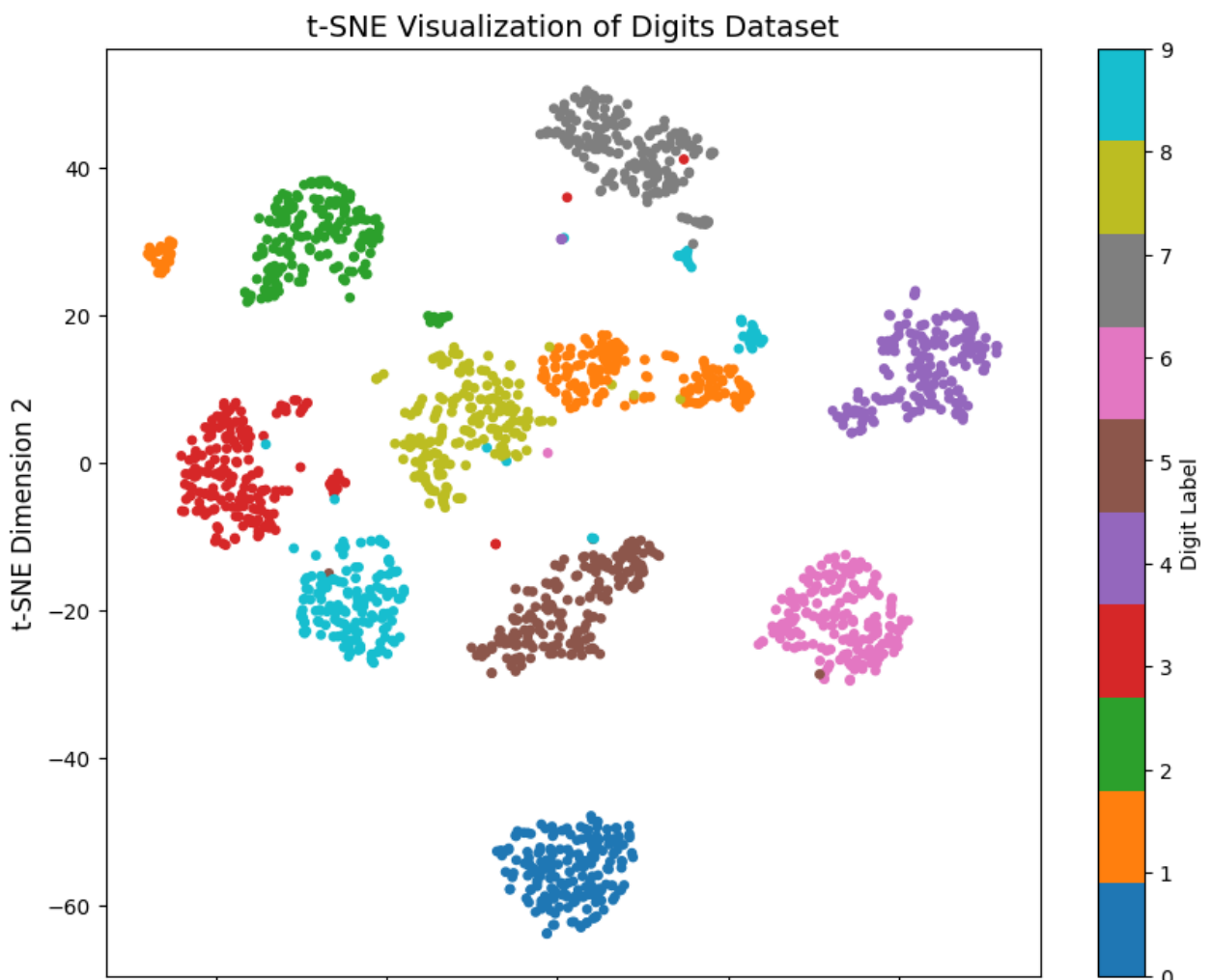
```
# Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.manifold import TSNE
```

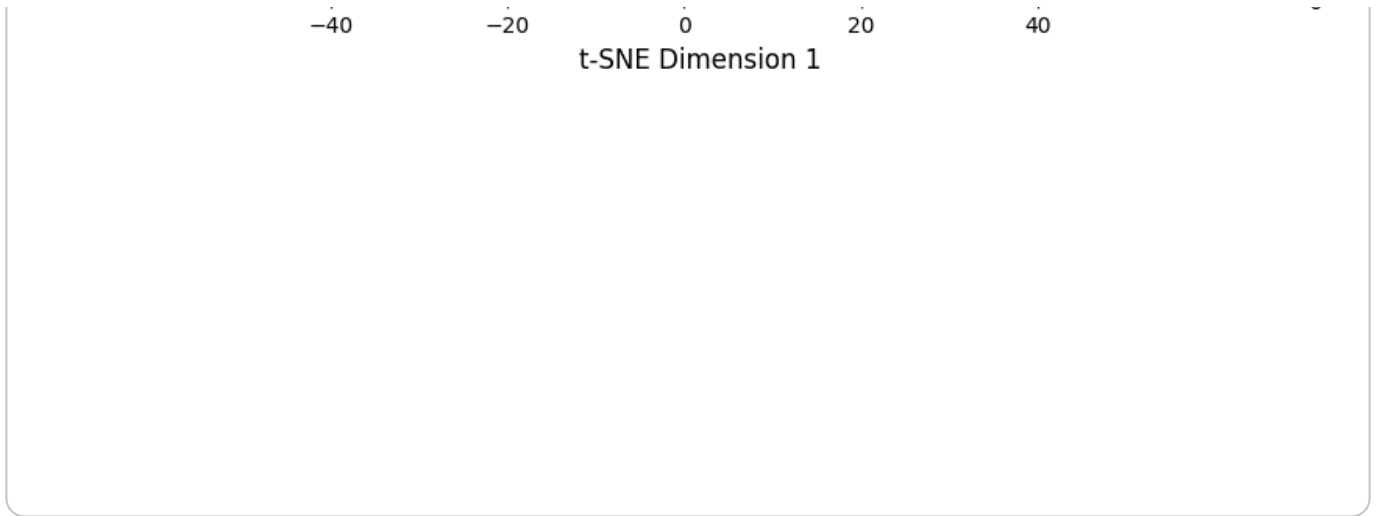


```
# Load the Digits dataset
digits = load_digits()
data = digits.data
labels = digits.target

# Apply t-SNE to reduce dimensions to 2D
tsne = TSNE(n_components=2, perplexity=30, random_state=42)
data_tsne = tsne.fit_transform(data)

# Plot the results
plt.figure(figsize=(10, 8))
scatter = plt.scatter(data_tsne[:, 0], data_tsne[:, 1], c=labels, cmap=
plt.colorbar(scatter, label="Digit Label")
plt.title("t-SNE Visualization of Digits Dataset", fontsize=14)
plt.xlabel("t-SNE Dimension 1", fontsize=12)
plt.ylabel("t-SNE Dimension 2", fontsize=12)
plt.show()
```





Explanation of the Example

1. **Dataset:** The Digits dataset, which consists of handwritten digit images (64 features per image), is used as input.
2. **t-SNE Application:** The algorithm reduces the dataset to 2D, emphasizing local relationships while clustering similar digits.
3. **Visualization:** The scatterplot shows well-separated clusters corresponding to different digit classes.

Comparison Table of Dimensionality Reduction Techniques

Criteria	PCA	LDA
Type	Linear (unsupervised)	Linear (supervised)
Goal	Maximize variance across components	Maximize class separability in components
Data Assumptions	Assumes linear relationships	Assumes linear separability of classes
Output	Principal components (orthogonal)	Linear discriminants
Strengths	- Captures maximum variance - Simple and fast	- Enhances class separability - Useful for classification
Weaknesses	- Loses interpretability - Linear only	- Requires labeled data - Assumes linear separability
Best Use Cases	- Exploratory data analysis - Preprocessing	- Classification tasks - Dimensionality reduction for supervised learning
Scalability	High scalability	Moderate scalability
Interpretability	Moderate	High
Computational Cost	Low	Low to Moderate
Key Parameters	Number of components	Number of components, class labels

Key Takeaways

- **PCA:** Ideal for linearly related data with no labels.
- **LDA:** Best for classification tasks with labeled data.
- **NMF:** Useful for interpretable decompositions (e.g., text, images).
- **LLE:** Captures local non-linear structures but sensitive to noise.
- **Isomap:** Combines local and global relationships; good for manifold learning.
- **t-SNE:** Excels in visualization, especially for complex, non-linear datasets.

